Lenguajes de Programación de IA. Lenguaje R

Caso práctico



Imagen generada por IA con Freepik (Licencia Freepik)

Ana es la directora de "Envíos Rápidos", una pequeña, pero creciente empresa que se dedica a gestionar los envíos y entregas para varias tiendas de moda en línea. Siempre apasionada por la tecnología y la eficiencia, Ana se dio cuenta de que la Inteligencia Artificial podría transformar su negocio y resolver las dificultades surgidas por el aumento de la demanda de sus clientes, llevando su empresa al siguiente nivel. Con esta visión en mente, montó un equipo de expertos en datos y se dispuso a iniciar la fase de desarrollo de su ambicioso proyecto.

Ana sabía que la clave del éxito residía en optimizar los procesos logísticos y mejorar la precisión en las entregas. Para ello, necesitaba analizar grandes volúmenes de datos históricos sobre los envíos: tiempos de entrega, rutas de los repartidores, incidencias, devoluciones y satisfacción del cliente.

Pero ¿por dónde empezar? El gran volumen de datos almacenados en distintos sistemas y formatos parecía abrumador, tenía que encontrar la forma de explotar todos esos datos internos en beneficio de su empresa.

Lenguajes de Programación para Inteligencia Artificial



Imagen de freepik en Freepik (Licencia Gratuita Freepik)

La inteligencia artificial (IA) ha transformado diversas industrias al permitir la automatización de tareas complejas, el análisis de grandes volúmenes de datos y el desarrollo de sistemas inteligentes que pueden aprender y adaptarse. Para implementar técnicas y algoritmos de IA, se utilizan varios lenguajes de programación. Entre los más destacados se encuentran Python, R y Java.

Python

Python es, sin duda, el lenguaje de programación más popular en el campo de la inteligencia artificial. Sus características son su simplicidad, legibilidad, versatilidad, una amplia biblioteca de herramientas y una gran comunidad de desarrollo.

R

R es otro lenguaje prominente en el ámbito de la Inteligencia Artificial, especialmente en análisis estadístico y minería de datos. Posee una gran fortaleza en el análisis estadístico y visualización de datos. Su gran oferta de paquetes especializados y su facilidad de integrarse con otros lenguajes facilitan la implementación de algoritmos de aprendizaje automático. En esta unidad se hace una introducción a este lenguaje en auge.

Java

Java es un lenguaje importante en el desarrollo de aplicaciones de inteligencia artificial. Aunque no es tan popular como Python en este campo. Sus fortalezas son su alto rendimiento y escalabilidad, su portabilidad y la facilidad de integración en aplicaciones empresariales. Además, proporciona Frameworks que permiten el desarrollo de modelos de IA.

Cada uno de estos lenguajes tiene sus propias fortalezas y es elegido según los requerimientos específicos del proyecto. Python es preferido por su simplicidad y extensas bibliotecas, R es elegido por su capacidad analítica y visualización de datos, y Java es valorado por su rendimiento y escalabilidad en aplicaciones empresariales.

1. El Lenguaje de Programación R

Caso práctico

Ana decidió que el **Lenguaje R** sería la herramienta principal para este análisis, dada su potencia para el manejo de datos y capacidades estadísticas.

El primer paso de Ana fue recopilar todos los datos almacenados en los sistemas de "Envíos Rápidos". Con la ayuda de su equipo, extrajo información de diversas fuentes, incluyendo bases de datos SQL, archivos CSV y registros internos. A continuación, comenzó a utilizar R para limpiar y organizar estos datos, asegurándose de que estuvieran listos para un análisis más profundo.

Ana y su equipo emplearon varios paquetes de R, como **dplyr** y **tidyr**, para manipular y transformar los datos. Identificaron patrones y tendencias en las entregas, como las horas pico de demanda y las rutas más eficientes. Utilizando **ggplot2**, crearon visualizaciones detalladas que les permitieron entender mejor el comportamiento de las entregas y detectar posibles cuellos de botella.

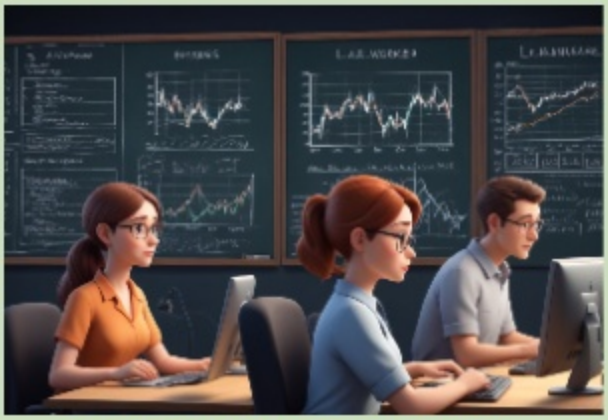


Imagen generada por IA con Freepik (Licencia Freepik)

Uno de los principales objetivos de Ana era reducir los tiempos de entrega y mejorar la precisión en las estimaciones. Con este fin, aplicaron técnicas de clustering utilizando el paquete **kmeans** de R para agrupar las áreas de entrega en clústeres basados en la proximidad geográfica y el volumen de pedidos. Esto les permitió diseñar rutas más optimizadas para los repartidores.

Además, Ana quería predecir las demandas futuras para gestionar mejor los recursos y personal. Su equipo implementó modelos de series temporales con el paquete **forecast** de R, que les permitió hacer predicciones precisas sobre la demanda en diferentes periodos. Esto fue crucial para planificar con antelación y asegurarse de que siempre hubiera suficiente personal y vehículos disponibles durante los picos de demanda.

Con los datos bien analizados y las predicciones en mano, Ana y su equipo empezaron a desarrollar algoritmos de aprendizaje automático que podrían integrarse en el sistema de gestión de envíos de la empresa. Estos algoritmos no solo sugerían las mejores rutas en tiempo real, sino que también optimizaban la asignación de pedidos a los repartidores, teniendo en cuenta factores como el tráfico y las condiciones meteorológicas.

El uso de R y la implementación de técnicas de Inteligencia Artificial transformaron radicalmente "Envíos Rápidos". Ana logró reducir significativamente los tiempos de entrega y aumentó la satisfacción del cliente. Además, la empresa pudo manejar un mayor volumen de pedidos sin necesidad de incrementar proporcionalmente los recursos, lo que se tradujo en un crecimiento sostenible y rentable.

Gracias a su visión y liderazgo, Ana no solo mejoró los procesos de su empresa, sino que también se posicionó a la vanguardia de la innovación en el sector logístico, demostrando que la Inteligencia Artificial, bien aplicada, puede ser una poderosa aliada para cualquier negocio.

¿Qué es R?

R es un lenguaje de programación y un entorno de software libre diseñado específicamente para el análisis estadístico y la visualización de datos. Fue desarrollado originalmente por Ross Ihaka y Robert Gentleman en la Universidad de Auckland, Nueva Zelanda, y ha sido mantenido y mejorado por la comunidad del R Project.

A través del entorno de R vamos a ser capaces de manipular datos, realizar análisis sobre esos datos y presentar los resultados gráficamente para facilitar su interpretación.

Scripts trabajados durante la Unidad



Estantería con varios paquetes. (CC0)

En el siguiente fichero comprimido encontrarás los scripts trabajados durante esta unidad.

En los contenidos se proporcionan capturas de pantalla del IDE RStudio, pero es mucho más útil ejecutar estos scripts línea a línea, mientras se observan las salidas por consola y el estado de las variables.

En el apartado 1.2 se explica como preparar el entorno de trabajo para R, una vez listo, se recomienda trabajar de forma paralela los contenidos de esta unidad y los scripts proporcionados con RStudio, para un óptimo aprovechamiento de la unidad. También es aconsejable realizar variaciones sobre el código para explorar la funcionalidad de R.

PIA02_Scripts.zip

1.1 Características de R

Características Principales de R

R es un lenguaje de programación de alto nivel, que permite realizar cálculos estadísticos complejos y manipulación de datos de manera eficaz. Proporciona un amplio conjunto de operadores para el cálculo de arrays y matrices, además de un eficiente sistema de almacenamiento.



Logo de R (CC BY-SA)

Es interpretado, lo que significa que se puede ejecutar línea por línea sin necesidad de compilación. Su intérprete está disponible para los sistemas operativos más populares.

R es gratuito y su código es abierto, lo que permite a cualquier usuario descargarlo, usarlo y modificarlo según sus necesidades sin costo alguno.

Posee una arquitectura extensible gracias a paquetes de funcionalidad específica, desde una amplia colección de paquetes disponibles a través de CRAN (Comprehensive R Archive Network), que extienden sus capacidades para cubrir una variedad de necesidades estadísticas y de análisis de datos, hasta repositorios como Bioconductor, que ofrece paquetes específicos para bioinformática.

Dispone de un sistema gráfico para el análisis y la visualización de datos. R es famoso por sus capacidades gráficas, permitiendo crear desde gráficos simples hasta visualizaciones complejas y personalizadas.

Tiene una alta capacidad de integración con otras tecnologías. Puede interactuar con distintas bases de datos y herramientas, permitiendo importar y exportar datos en diversos formatos (CSV, Excel, SQL, etc.). También puede integrarse con otros lenguajes de programación como Python y C++ para mejorar la funcionalidad y eficiencia.

Incluye una amplia gama de técnicas estadísticas, como modelado lineal y no lineal, pruebas estadísticas, análisis de series temporales, clasificación, clustering y más. Los resultados de los análisis pueden ser interpretados directamente en R y exportados a diferentes formatos para reportes y publicaciones.

¿Qué es CRAN?

CRAN es el *The Comprehensive R Archive Network*, una red en la que se archivan todas las versiones de R base, así como todos los paquetes para R que han pasado por un proceso de revisión riguroso, realizado por el CRAN Team, que se encarga de asegurar su correcto funcionamiento.

CRAN es una red porque existen copias de su contenido en diferentes servidores alrededor del mundo, los cuales se actualizan diariamente. De este modo, no importa de qué servidor de CRAN descargues R o algún paquete, lo que vas a obtener será la versión más reciente de ese recurso, que es igual a la disponible en todos los demás servidores.

Autoevaluación

1) ¿Cómo se ejecuta R?

- ○ R es un lenguaje compilado, es necesario compilar el código fuente y obtener un archivo ejecutable.
- ○ R es un lenguaje interpretado, sus sentencias se ejecutan por un intérprete. Lo hace que sea portable.
- ○ R es un lenguaje semicompilado, combina características de los lenguajes compilados e interpretados.
- ○ R es un lenguaje precompilado. Antes de ejecutarse pasa por una fase de compilación que traduce el código fuente a un código intermedio.

Para saber más

R es un lenguaje que está ganando popularidad, puedes encontrar un gran número de manuales y cursos para profundizar.

Aquí algunos enlaces interesantes:

Manuales Oficiales.

Web de Consulta y Manuales.

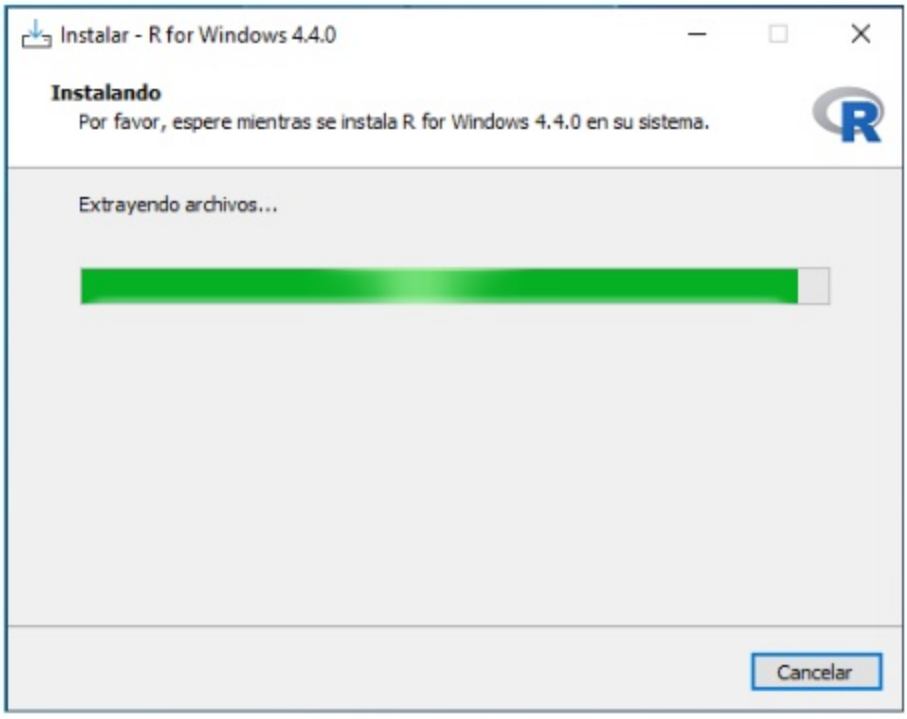
Tutorial de R.

1.2 Entorno de Desarrollo

Instalación del Software

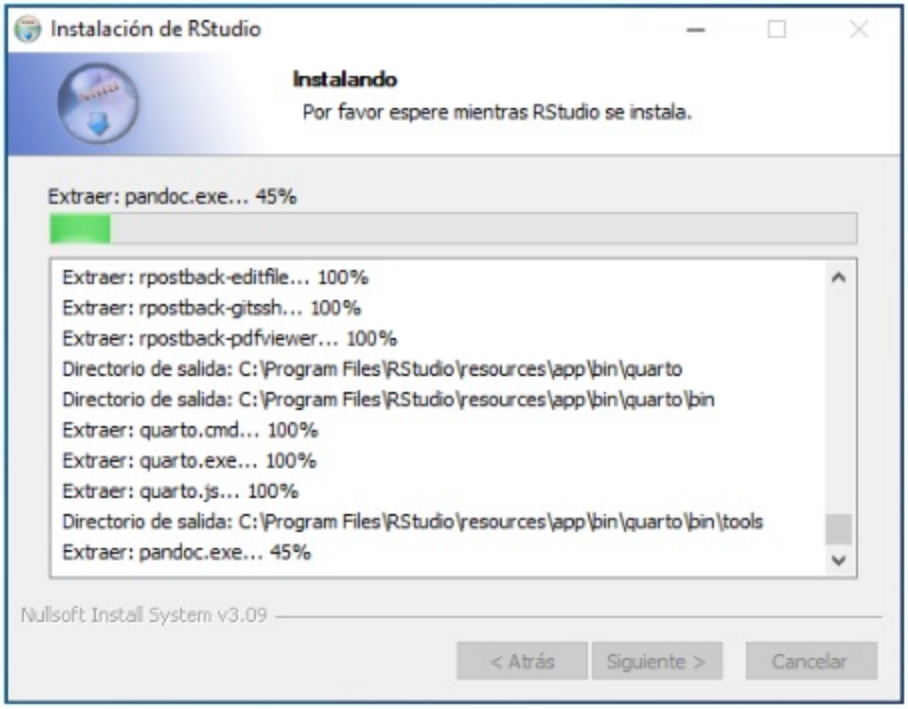
Para empezar a usar R, debemos instalar tanto el Intérprete de R como el entorno de desarrollo de RStudio.

Para instalar el intérprete visitaremos el sitio oficial de R (R-project), pulsamos en Descargar R, seleccionamos el servidor desde el que descargarlo, preferiblemente uno ubicado en España, y descargamos el instalador correspondiente al sistema operativo que nos interese (Windows, macOS o Linux). Ejecutaremos el instalador y seguiremos las instrucciones para completar la instalación.



(Elaboración propia). *Captura de la Instalación del Intérprete R*

Una vez instalado el intérprete, instalaremos el entorno de RStudio, para ello visitaremos el sitio oficial de RStudio (https://posit.co/products/open-source/rstudio/) y descargaremos el instalador correspondiente al sistema operativo, igualmente seguiremos las instrucciones del instalador para completar la instalación.



(Elaboración propia). *Captura de la Instalación de RStudio*

Entorno de RStudio

1. Barra de Herramientas

La encontramos en la parte superior de la ventana. Contiene el menú y accesos directos a funciones y herramientas comunes. Por ejemplo, botones de ejecución (para ejecutar código seleccionado, ejecutar una línea, ejecutar el script completo o detener la ejecución).

2. Editor de Código

Se encuentra en la parte central izquierda y ocupa la mayor parte de la ventana. Aquí es donde se escribe y edita el código R. Este editor colorea el código para mejorar la legibilidad, numera las líneas para ayudar a localizar errores y a referenciar partes específicas del código, e incluso tiene funciones de autocompletado (realiza sugerencias de código mientras escribes).

3. Console / Terminal / Background Jobs

Generalmente, se encuentra en la parte inferior izquierda.

- La Consola es el lugar donde se ejecuta el código R en tiempo real. Puedes introducir comandos directamente y ver los resultados inmediatamente. También puedes acceder a comandos previamente ejecutados usando las teclas de flecha hacia arriba y hacia abajo.
- Además encontramos la Terminal, donde podemos lanzar comandos del SO y los Procesos Secundarios que permite ejecutar trabajos en segundo plano.

4. Environment / History

Se encuentra en la parte superior derecha. Proporciona una vista del entorno actual de trabajo y el historial de comandos ejecutados.

- A través del Entorno podemos visualizar las variables, datos y objetos disponibles en el entorno de trabajo.
- El Historial contiene una lista de todos los comandos que has ejecutado durante la sesión. Puedes reutilizar estos comandos en la consola o el editor de scripts.

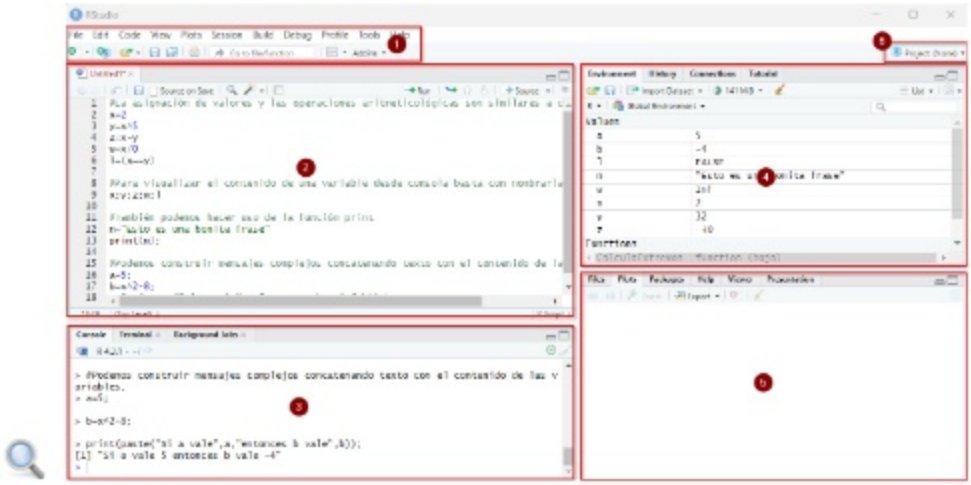
5. Files / Plots / Packages /Help / Viewer

Este panel multifuncional se encuentra en la parte inferior derecha. Tiene varias pestañas que permiten realizar diferentes tareas:

- Files: Navegador de archivos del sistema. Permite abrir, mover y gestionar archivos en tu directorio de trabajo.
- Plots: Muestra los gráficos generados por el código R. Puedes exportar, copiar y borrar gráficos desde aquí.
- Packages: Lista y gestiona los paquetes de R instalados. Puedes cargar, actualizar y desinstalar paquetes desde esta pestaña.
- Help: Proporciona acceso a la documentación de R. Puedes buscar ayuda sobre funciones y paquetes específicos.
- Viewer: Visualiza salidas de HTML y otros formatos generados por documentos R Markdown y aplicaciones Shiny.

6. Panel de Proyecto

Se encuentra en la esquina superior derecha de la ventana. Permite abrir, cerrar y gestionar proyectos. Los proyectos en RStudio ayudan a organizar el trabajo y mantener un entorno de trabajo específico para cada proyecto.



(Elaboración propia). *Captura del Entorno de RStudio*

Autoevaluación

2) ¿Qué pestaña del panel inferior derecho se utiliza para visualizar gráficos generados por el código R?

- Viewer.
- Plots.
- Files.
- Packages.

3) ¿Cuál de las siguientes opciones no es una característica del Editor de código en RStudio?

- Numeración de líneas.
- Autocompletado.
- Historial de Comandos.
- Resaltado de sintaxis.

1.3 Tipos de variables y operaciones

R dispone de toda la gama de variables básicas. Además ofrece una gran potencia en el manejo estructuras de datos como vectores, matrices y dataframes, a la vez que dispone de unas librerías muy versátiles.

En cuanto al manejo de datos:

- **Datos básicos:** numérico, lógico, fecha y cadena. Además de otros muy útiles como los factores.
- **Inferencia de tipos:** el intérprete determina el tipo de una variable en tiempo de ejecución.
- **Tipado dinámico:** las variables pueden cambiar su tipo en tiempo de ejecución.
- **Tipado fuerte:** no se permiten operaciones entre tipos de datos incompatibles. Para cambiar el tipo de datos es preciso utilizar determinadas funciones.

Comentarios

En R, como en cualquier otro lenguaje, los comentarios se utilizan para agregar notas y descripciones dentro del código, lo que ayuda a mejorar la legibilidad y la comprensión del código por parte de otros programadores (o de ti mismo en el futuro).

Los comentarios en R se agregan utilizando el símbolo **#**. Todo lo que sigue al **#** en una línea es tratado como un comentario y no se ejecuta como código.



Escritorio (CC0)

Buenas Prácticas para Comentarios

- **Claridad y Concisión:** Los comentarios deben ser claros e ir a lo importante. En esta unidad se utilizan línea a línea para clarificar las instrucciones de ejemplo, pero no es lo recomendable, solo deben comentarse las partes más importantes y complejas del código.
- **Actualización:** Conforme se realizan modificaciones del código, es preciso revisar y actualizar los comentarios, de esta forma nos aseguramos de su vigencia y no añaden confusión al código.
- **Descriptivos:** Los comentarios deben describir el "qué" y el "por qué" del código, más que el "cómo". El "cómo" debe ser evidente a partir del propio código, debes tener en cuenta que los comentarios están destinados a programadores, ya conocen el lenguaje, solo necesitan saber qué hace el código.

Autoevaluación

4) ¿Qué característica de R permite que las variables puedan cambiar su tipo en tiempo de ejecución?

- Tipado fuerte.
- Tipado dinámico.
- Inferencia de tipos.
- Tipado débil.

Debes conocer



Símbolo igual (CC0)

En R, tanto **=** como **<-** se pueden utilizar para asignar valores a variables, pero hay algunas consideraciones sobre cuándo y cómo se deben utilizar cada uno. La recomendación general es usar <- para la asignación de variables y reservar = para dar valor a los argumentos de funciones. Esto no solo sigue las convenciones de estilo comúnmente aceptadas en R, sino que también ayuda a evitar confusiones y errores en el código.

En resumen:

Usa <- para la mayoría de las asignaciones de variables.Usa = principalmente para asignar valores a los argumentos en las funciones.

Ejemplo:

```
media <- mean(x = c(1, 2, 3, 4, 5))
```

1.3.1 Tipos de Variables

R soporta varios tipos de variables básicos, cada uno adecuado para diferentes tipos de análisis y manipulación de datos. Los tipos de variables más comunes:



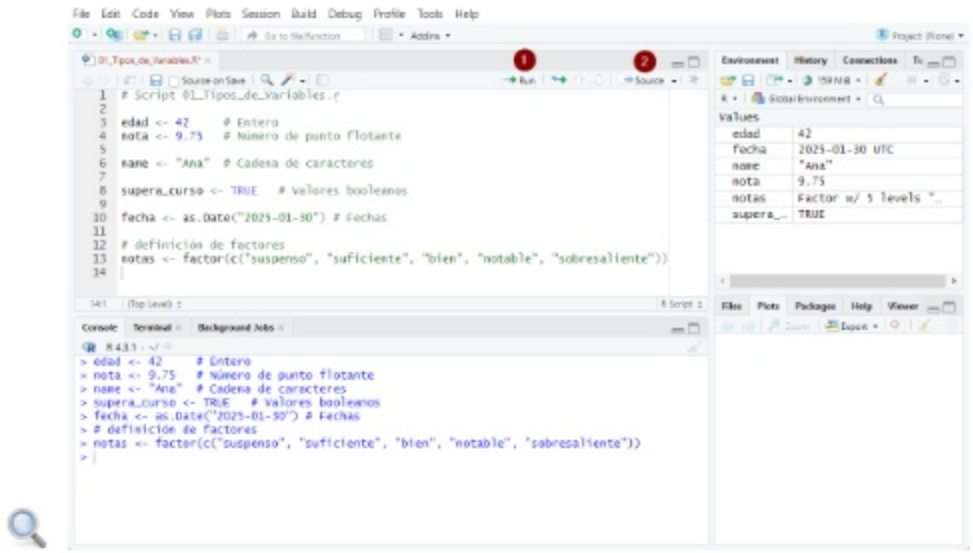
Taquillas (CC0)

- **Numéricos:** Pueden ser enteros sin decimales o en punto flotante (números con decimales).
- **Cadenas de caracteres:** Representan secuencias de caracteres, es decir, texto. Las cadenas de texto se expresan entre comillas.
- **Lógicos:** Representan valores booleanos, es decir, TRUE o FALSE.
- **Fechas y horas:** Podemos definir fechas y horas a partir de cadenas bien formadas. Y aplicarles formato según nos interese.
- **Factores:** Representan datos categóricos y son útiles para variables cualitativas, que pueden tomar unos determinados valores. Este tipo de variable resulta muy útil en combinación con otras estructuras como los dataframes. Su uso se explicará en profundidad más adelante.

Ejemplo:

A continuación veremos una captura de la ejecución de un pequeño script de ejemplo (**01_Tipos_de_Variables.R**) donde definimos varias variables de distintos tipos. Para ejecutar el Script podemos hacerlo línea a línea (1) o todo el script de una vez (2).

Observa como el código del script se encuentra en la zona central y está formateado, se distinguen claramente variables, literales y comentarios. En la ventana inferior de **Consola** podemos ver la ejecución línea a línea del código (si se produjera algún error nos lo notificaría en esta zona) y arriba a la derecha podemos ver el **Entorno** donde se visualiza el estado de las variables y otros objetos, mientras se ejecuta el código puedes ver como se modifica el contenido de las variables.



(Elaboración propia). *Captura de RStudio*

Autoevaluación

5) ¿Qué tipo de variable debería utilizarse para guardar el estado de la entrega de un paquete que puede tomar los valores: "Preparado", "En Transito" o "Entregado"?

- Numérica.
- Lógica.
- Fecha y hora.
- Factor.

1.3.2 Operaciones Básicas

R permite realizar todo tipo de operaciones básicas sobre las variables:

Operaciones Aritméticas

- Suma: **+**
- Resta: **-**
- Multiplicación: *
- División: */*
- Potencia: **^**

Operaciones de Comparación

- Igual a: **==**
- No igual a: **!=**
- Mayor que: **>**
- Menor que: **<**
- Mayor o igual que: **>=**
- Menor o igual que: **<=**

Operaciones Lógicas

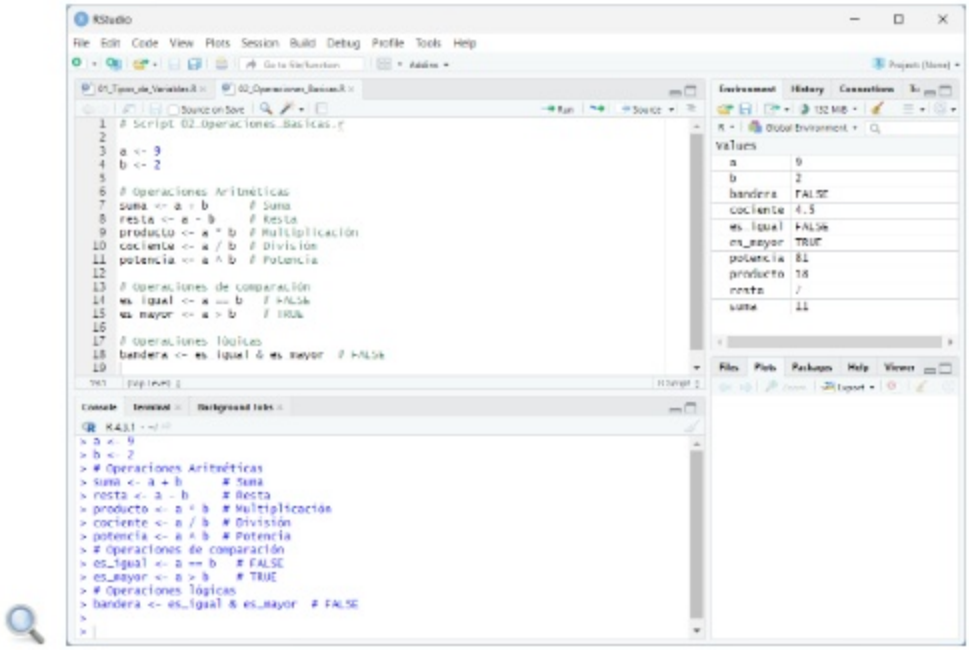
- AND: **&**
- OR: **|**
- NOT: **!**



Ábaco de colores (CC0)

Ejemplo:

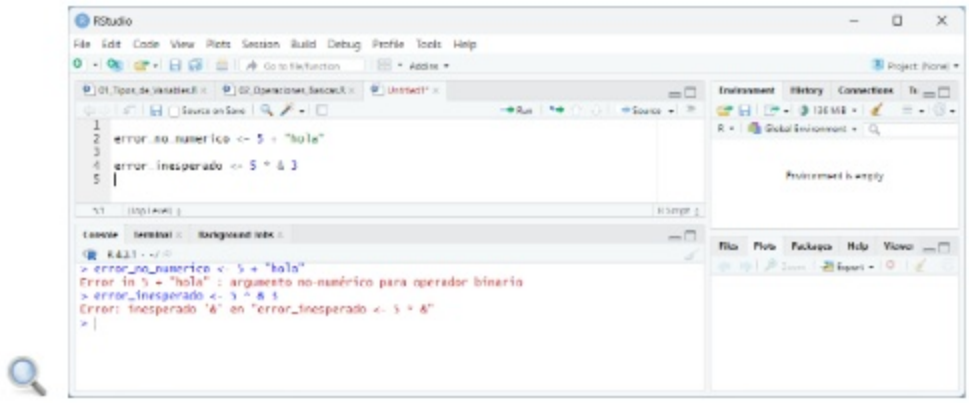
A continuación veremos una captura de la ejecución de un script de ejemplo (**02_Operaciones_Basicas.R**) donde haremos uso de los operadores descritos anteriormente.



(Elaboración propia). *Captura de RStudio*

¿Qué pasa si se produce un error?

Si al ejecutar una línea se produce un error, el intérprete nos informará de la causa a través de la Consola. Además no producirá ningún efecto en el Entorno, como puedes ver en el siguiente ejemplo, no aparecen las variables en el entorno, ya que no han llegado a crearse.



(Elaboración propia). *Captura de RStudio*

1.3.3 Conversión de Tipos

En R, la conversión de tipos de datos es una operación común y necesaria debido a la naturaleza del análisis de datos, donde los datos pueden provenir de diversas fuentes y en distintos formatos. Por ejemplo, los datos importados desde archivos CSV pueden llegar como cadenas de caracteres, pero para realizar cálculos matemáticos, es necesario convertir estos datos a numéricos.

La conversión de tipos de datos es fundamental para asegurar que los datos se manipulen y analicen correctamente. Cada tipo de dato tiene sus propias características y usos, y la capacidad de convertir entre tipos permite a los programadores de R aprovechar al máximo las herramientas y funciones disponibles en este lenguaje.



Ordenador con Gráficos (CC0)

R proporciona varias funciones para realizar estas conversiones de manera explícita:

as.numeric(): Convertir a Numérico
Esta función convierte un objeto a un tipo de dato numérico. Es útil cuando se necesita realizar operaciones matemáticas o análisis estadísticos. Si el valor no se puede convertir a numérico, as.numeric() devolverá NA con una advertencia.

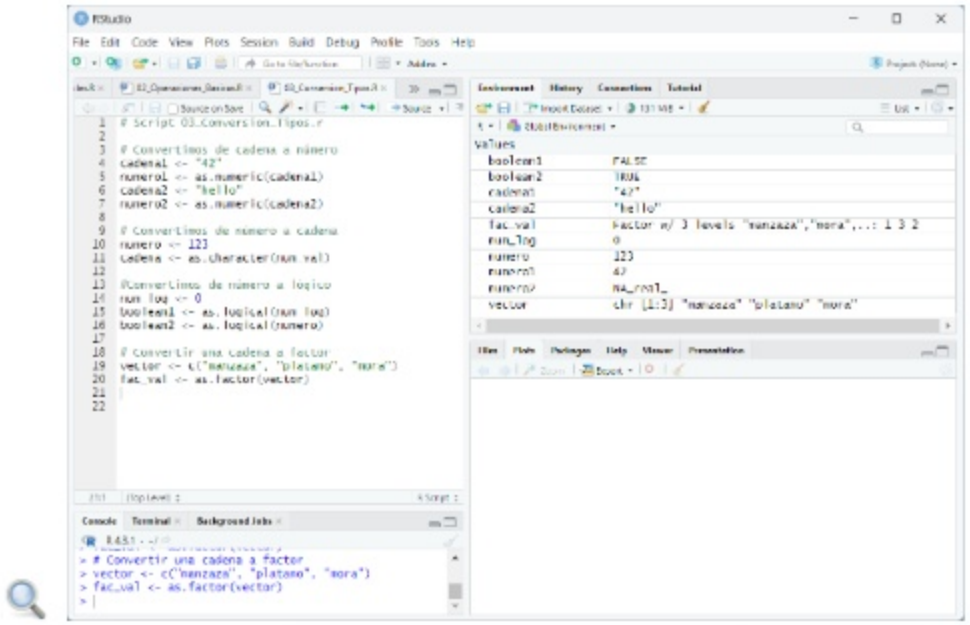
as.character(): Convertir a Cadena
Esta función convierte un objeto a una cadena de caracteres. Es útil cuando se necesita manipular o imprimir datos como texto.

as.logical(): Convertir a Lógico
Esta función convierte un objeto a un tipo de dato lógico (TRUE o FALSE). Es útil para evaluar condiciones y realizar operaciones lógicas. Cualquier valor numérico distinto de cero se convierte en TRUE.

as.factor(): Convertir a Factor
Esta función convierte un objeto a un factor, que es útil para el análisis de datos categóricos. Los factores son esenciales para ciertos tipos de análisis estadísticos y gráficos en R.

Ejemplo

A continuación veremos una captura de la ejecución de un script de ejemplo (**03_Conversion_Tipos.R**) donde haremos uso de las funciones descritas anteriormente. Son casos sencillos, pero estas funciones aceptan parámetros que modifican su comportamiento. Si quieres profundizar puedes consultar los enlaces de manuales proporcionados en el apartado 1.1.



(Elaboración propia). *Captura de RStudio*

Observa como en este ejemplo la cadena "42" se transforma en el número 42, en cambio, la cadena "hello" no tiene correspondencia con un número por lo que la variable numero2 toma el valor **NA_real_** (Valor faltante - "Not Available"), en este caso el intérprete infiere que se trataría de un valor de tipo real, por ser la salida de la función as.numeric().

Sobre la función as.logical() podemos ver que el 0 lo convierte en FALSE y cualquier otro número (en el ejemplo 123) lo convierte en TRUE.

Importancia de NA en Análisis de Datos

La presencia de NA (No Disponibles) en los datos es común y su manejo adecuado es crucial en el análisis de datos. Ignorar los NA puede llevar a resultados sesgados o incorrectos. Por lo tanto, es esencial comprender cómo identificar, eliminar o imputar valores faltantes para asegurar la calidad y la integridad del análisis.

En resumen, NA en R es una herramienta esencial para representar y manejar datos faltantes, permitiendo a los analistas y científicos de datos tratar adecuadamente las situaciones en las que la información no está disponible.

Autoevaluación

6) ¿Qué tipo de análisis requiere la conversión de datos a factores?

- Análisis de datos numéricos.
- Análisis de datos categóricos.
- Análisis de series temporales.
- Análisis de datos lógicos.

7) ¿Qué sucede si as.numeric() no puede convertir un valor a numérico?

- Devuelve un valor de 0.
- Devuelve una cadena de caracteres vacía.
- Devuelve FALSE.
- Devuelve NA con una advertencia.

1.3.4 Operaciones sobre Cadenas

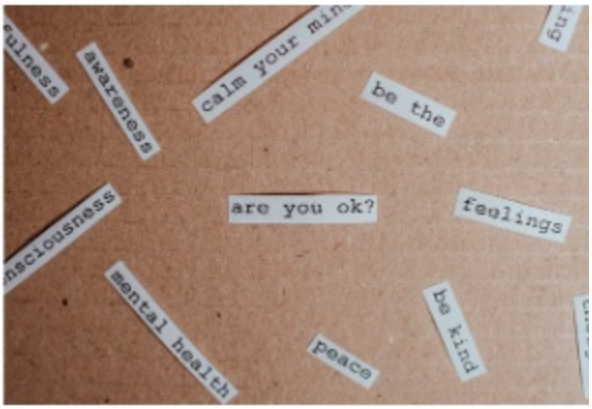
En R, se puede realizar una amplia variedad de operaciones con cadenas de caracteres. Estas operaciones te permitirán manipular y analizar cadenas de manera eficiente, facilitando tareas como la limpieza de datos, la generación de informes y la preparación de datos para análisis más avanzados.

Concatenación de Cadenas

- **paste():** Une cadenas permitiendo indicar un separador, por defecto será un espacio en blanco.
- **paste0():** No utiliza separador, une directamente las cadenas.

Subconjuntos de Cadenas

- **substr()**: Extrae una subcadena de una cadena dada.



Recortes de frases y palabras (CC0)

Cambio entre mayúsculas y minúsculas

- **toupper():** Convierte toda la cadena a mayúsculas.
- **tolower():** Convierte toda la cadena a minúsculas.

Búsqueda y Reemplazo

- **grep():** Busca patrones en cadenas y devuelve las posiciones de las cadenas donde se encuentran. Esta operación se suele realizar sobre conjuntos de datos, por ejemplo vectores (que estudiaremos más adelante).
- **grepl():** Devuelve un vector lógico indicando si el patrón se encuentra en cada elemento del vector de entrada.
- **sub():** Sustituye patrones en cadenas. Reemplaza solo la primera coincidencia.
- **gsub():** También sustituye patrones en cadenas, pero reemplaza todas las coincidencias.

División de Cadenas

- **strsplit():** Divide una cadena en partes, basándose en un delimitador.

Longitud de Cadenas

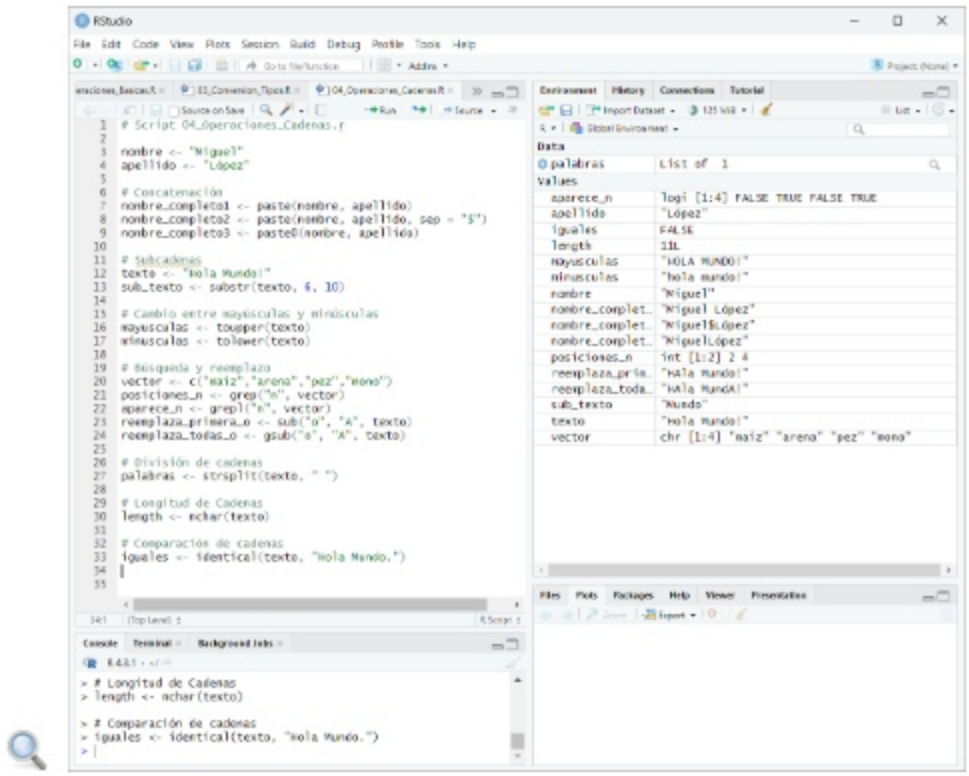
- **nchar():** Devuelve la longitud de una cadena.

Comparación de Cadenas

- **identical():** Compara cadenas. Devuelve TRUE si son iguales.

Ejemplo

Aquí puedes ver algunos ejemplos de cómo se aplican estas funciones, es mucha información que procesar, es recomendable ejecutar línea a línea el script proporcionado (**04_Operaciones_Cadenas.R**) y ver como van apareciendo las variables en el entorno y los valores que van tomando.



(Elaboración propia). *Captura de RStudio*

Casi todas las funciones sobre cadenas tienen un comportamiento previsible, las que pueden resultar algo más confusas pueden ser grep y grepl.

Con **grep** hemos buscado en qué elementos (cadenas) del vector aparece la letra "n", como resultado nos indica que el patrón "n" se encuentra presente en las cadenas 2 y 4.

Con **grepl** hemos realizado la misma búsqueda, pero nos proporciona la salida en distinto formato, la salida es un vector de valores lógicos: FALSE, TRUE, FALSE y TRUE. Que quiere decir que en las cadenas que ocupan las posiciones 1 y 3 no aparece la letra "n" y en las cadenas que ocupan las posiciones 2 y 4 si que aparece la letra "n".

Autoevaluación

8) ¿Cuál es la función en R que une cadenas sin utilizar un separador?

- paste()
- paste0()
- substr()
- strsplit()

9) ¿Cuál es la función en R que reemplaza todas las coincidencias de un patrón en una cadena?

- sub()
- grep()
- grepl()
- gsub()

1.3.5 Operaciones sobre fechas/horas

Trabajar con fechas y horas es esencial para muchos análisis de datos, especialmente en áreas como la econometría, los estudios de mercado y el análisis de series temporales. R proporciona varias funciones y paquetes para manipular y analizar datos de fechas y horas.



Imagen generada con la IA de Freepik (Licencia Gratuita Freepik)

1) Creación de Fechas y Horas

- **as.Date():** Funciona como los conversores vistos anteriormente, convierte una cadena de texto a un objeto de fecha.
- **as.POSIXct(), as.POSIXlt():** Convierten una cadena de texto a un objeto de fecha y hora.

2) Formateo de fechas y horas

- **format():** Extrae componentes específicos de fechas y horas.
- **weekdays(), months(), quarters():** Obtienen el nombre del día de la semana, el nombre del mes y el trimestre, respectivamente.

3)Tomar la fecha del sistema

- **Sys.Date():** Con esta función podemos tomar la fecha del sistema, es muy útil para tomar referencias de marcas de tiempo.
- **Sys.time():** Toma la fecha y hora del sistema en formato largo.

4) Operar con fechas

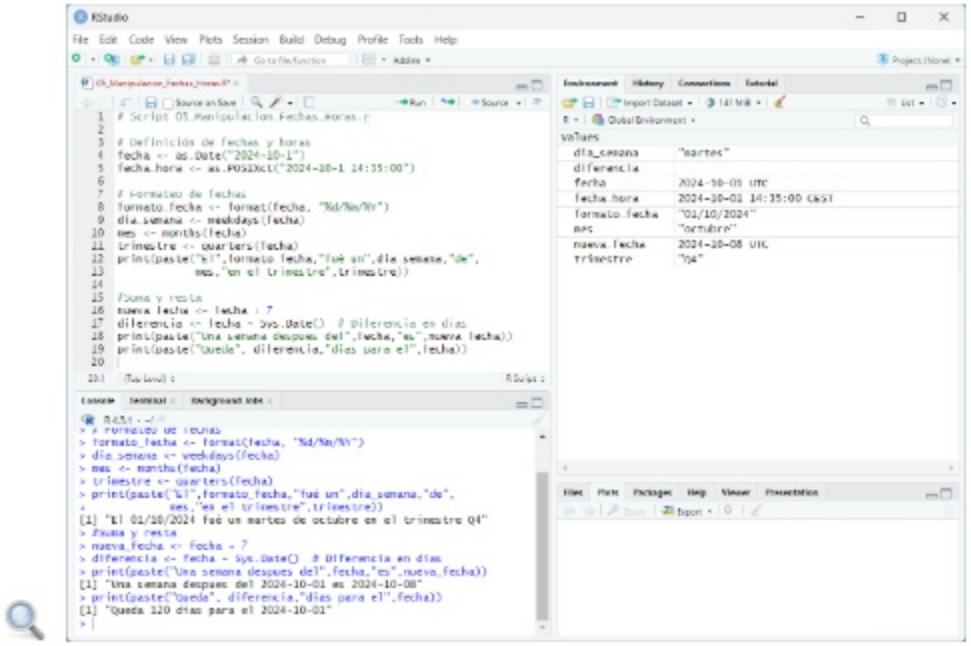
- **Suma y Resta de días:** Sumando o restando un entero a una fecha podemos incrementar o decrementar la fecha en un determinado número de días.
- **Diferencia entre fechas:** Igual ocurre restando fechas, obtenemos la diferencia en días.

5) Funciones de Redondeo – Utilizamos la librería lubridate

- **floor_date(), ceiling_date(), round_date():** Redondean fechas al inicio, final o al punto más cercano de un periodo específico.

Ejemplo:

En la siguiente captura podemos ver la ejecución del script **05_Manipulacion_Fechas_Horas.R** donde vemos algunos ejemplos de cómo se definen las fechas/horas y operaciones con este tipo de datos.



(Elaboración propia). *Captura de RStudio*

Observa la última línea de este script, la función print(), que veremos en el próximo apartado, muestra por consola el valor de un determinado objeto, en este caso es una cadena formada con la función paste() a partir de la composición de textos, una variable numérica y otra de tipo fecha.

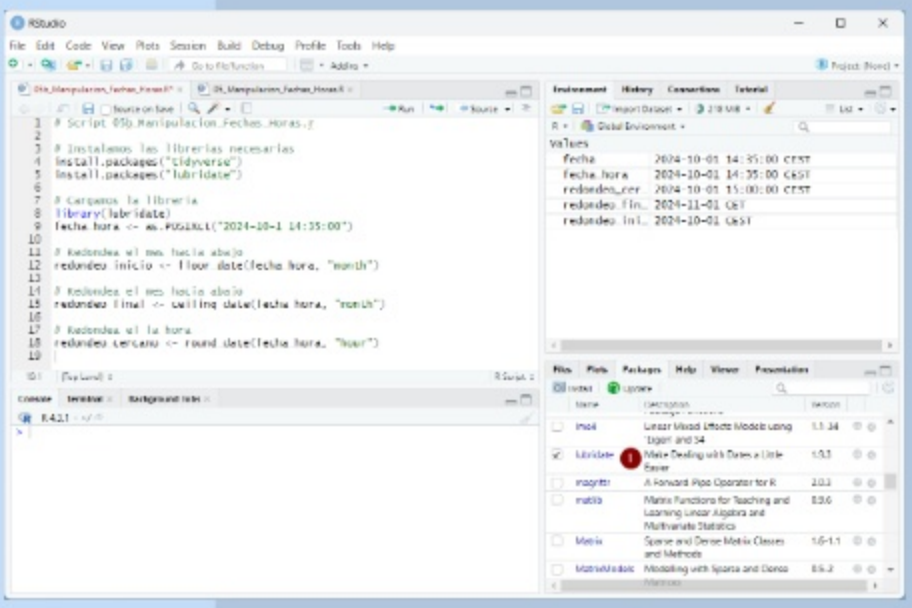
En R, las librerías son colecciones de funciones y datos que amplían las capacidades del lenguaje. Para instalar una librería, se utiliza la función **install.packages()**, proporcionando el nombre del paquete entre comillas.

Por ejemplo, para instalar el paquete lubridate que necesitamos para redondear fechas, se usa install.packages("lubridate").

Una vez instalado, el paquete debe ser cargado en el entorno de trabajo con la función **library()**, de la siguiente manera: library(lubridate), ya sin comillas.

Esto hace que todas las funciones y datos del paquete estén disponibles para su uso en el script, durante la sesión de R. Es importante instalar un paquete solo una vez, pero debe cargarse en cada nueva sesión de trabajo.

Veamos un ejemplo, instalaremos y cargaremos la librería lubridate. Después redondearemos algunas fechas para ver cómo funciona. Observa (1) que en el cuadrante de **Packages**, después de ejecutar el script, aparece la librería instalada y cargada (check).



(Elaboración propia). *Captura de RStudio*

También podrás encontrar el script correspondiente a esta captura en el paquete proporcionado. Puedes ver su nombre en el editor de código (**05b_Manipulacion_Fechas_Horas.r**), aparece en rojo porque se ha modificado después de guardarlo, de esta forma RStudio nos avisa de que la copia con la que estamos trabajando no está guardada.

Autoevaluación

10) ¿Cuál de las siguientes funciones se utiliza para obtener el día de la semana de una fecha en R?

- dia()
- months()
- weekdays()
- nomday()

11) ¿Qué función en R se utiliza para tomar solo la fecha del sistema?

- Sys.time()
- Sys.Date()
- as.Date()
- as.POSIXdto()

1.4 Mensajes por Consola



Composición de dígitos estilo Matrix (CC0)

En R, los mensajes de salida pueden utilizarse para visualizar los resultados de los cálculos o incluso informar sobre la ejecución del script (sucesos de advertencia o error). Usar estas funciones de manera adecuada ayuda a depurar y controlar la ejecución de los scripts de R.

print(): Imprime la representación de un objeto en la consola. Es la forma más básica de generar una salida. Además, se mostrará la información de indexación de los elementos del objeto, por líneas.

cat(): Concatena y muestra uno o más objetos, pero sin la información de indexación.

message(): Genera un mensaje en la consola, útil para mensajes informativos durante la ejecución de scripts. Estos mensajes pueden ser suprimidos o redirigidos.

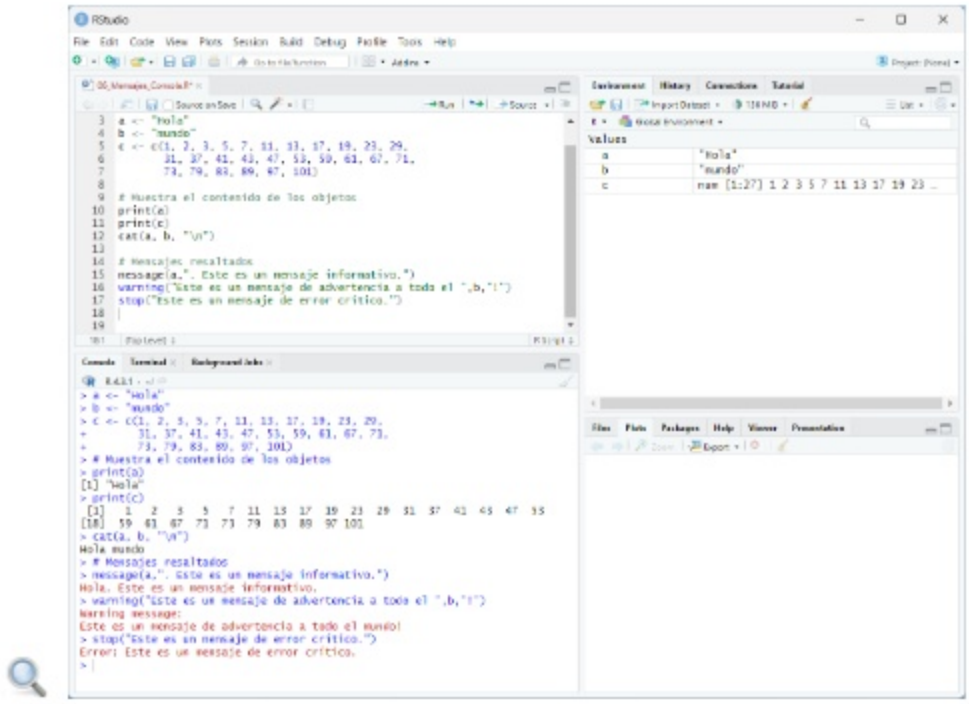
Warning(): Esta función genera un mensaje de advertencia y continúa la ejecución del script. Es útil para notificar al usuario sobre posibles problemas sin detener la ejecución.

Stop(): Esta función genera un mensaje de error y detiene la ejecución del script. Es útil para manejar errores críticos donde la ejecución no puede continuar.

Ejemplo

En el siguiente ejemplo (**06_Mensajes_Consola.r**) puedes ver como se muestran algunos tipos de mensajes por la consola de R, observa que las funciones message(), warning() y stop() tiene su salida formateada en rojo.

Sobre la función print, observa como indexa la salida en función de la longitud de la línea, el elemento [1] sería el número 1 y el elemento [18] de la salida sería el número 59.



(Elaboración propia). *Captura de RStudio*

Autoevaluación

12) ¿Cuál es la función en R que imprime la representación de un objeto en la consola y muestra la información de indexación de los elementos?

- cat()
- message()
- print()
- stop()

Ejercicio Resuelto

El siguiente script (**05c_Manipulacion_Fechas_Horas.R**) producirá una salida por consola:

```
ahora <- as.POSIXct(Sys.time())

ahora_r <- round_date(ahora,"hour")

message(format(ahora_r,"%A, %d de %B de %Y a las %H:%M horas"))
```

¿Qué salida generará? ¿Cómo será la salida?

Mostrar retroalimentación

¿Podrías reescribir el código en una sola línea?

Mostrar retroalimentación

1.5 Estructuras de Datos

En este apartado estudiaremos las estructuras de datos, que son fundamentales para el almacenamiento y manipulación masiva de gran cantidad de datos. Veamos con ejemplos algunas de las estructuras en R, aunque en posteriores apartados detallaremos el uso de las más relevantes.



Contenedores y Grúas en el Canal de Panamá (CC0)

Vectores: Son estructuras multidimensionales y homogéneas, es decir, todos los elementos deben ser del mismo tipo (numérico, carácter, lógico, etc.). Se crean con la función array().

Array unidimensional de números enteros construido por la concatenación de números, utilizando la función c().

```
vector1 <- array(c(1, 1, 2, 3, 5, 8, 13))
```

Array de tres dimensiones 2x2x2 que contiene los números del 1 al 8. También podemos utilizar la función c() para definir las dimensiones del vector.

```
vector2 <- array(1:8, dim = c(2, 2, 2))
```

Matrices: Son estructuras bidimensionales y homogéneas, todos los elementos deben ser del mismo tipo. Se crean con la función matrix().

Matriz de dos filas y tres columnas, que contiene los números del 1 al 12 de dos en dos.

```
matriz <- matrix(seq(1, 12, 2), nrow = 2, ncol = 3)
```

Listas: Son estructuras heterogéneas, pueden contener diferentes tipos de datos o incluso otras estructuras de datos. Se crean con la función list().

Lista de objetos de distinta naturaleza, este tipo de estructura permite nombrar sus elementos. En el ejemplo tenemos una estructura formada por tres objetos, un vector de números, un vector de letras y una matriz.

```
lista <- list( array = vector1, char = c("a", "b", "c"), matrix = matriz )
```

Data Frames: Son las estructuras más versátiles, son bidimensionales y heterogéneos, pueden contener diferentes tipos de datos en diferentes columnas. Se crean con la función data.frame().

Dataframe de nombres y edades, de tres filas y tres columnas, la primera columna será un identificador (id) que irá de 1 a 3, la segunda serán los nombres (name) y la tercera serán las edades (age). La forma más básica para construir un dataframe es indicar los elementos que lo contendrán, columna a columna.

dataframe <- data.frame(id = 1:3, name = c("Tolomeo", "Rufina", "Segismundo"), age = c(45, 60, 52))

En el script "**07_Estructuras_Datos.R**" puedes encontrar estas definiciones de ejemplo comentadas. También se muestra por consola el contenido de las estructuras, es interesante cotejar el contenido de las variables con la salida formateada ofrecida por print() para una mejor comprensión de las estructuras.

Como habrá observado, en estos ejemplos (y en algunos anteriores) se ha hecho uso de concatenaciones de objetos con c() y secuencias de números con a:b. Estas son construcciones auxiliares muy utilizadas en la definición e indexación de estructuras de datos. Ha llegado el momento de estudiar su funcionamiento.

Secuencias



Imagen de Freepik (Licencia Gratuita Freepik)

Una secuencia es una relación ordenada de objetos. Constituyen una herramienta poderosa para generar series de valores que podemos utilizar para construir estructuras más complejas o incluso acceder a determinados elementos de dichas estructuras. Veamos algunas formas de generar secuencias:

Operador ":" : con la sentencia a:b, donde a y b son números enteros, generamos una secuencia de enteros desde a hasta b. Esta es la forma más básica de generar secuencias de números enteros. Por ejemplo:

3:12 → Genera la secuencia: 3 4 5 6 7 8 9 10 11 12

100:95 → Genera la secuencia: 100 99 98 97 96 95

Función seq(): con esta función generamos una secuencia de números en un intervalo, acepta varios parámetros que modifican su comportamiento. Por ejemplo:

seq(1,10) → Genera la secuencia de 1 hasta 10, ambos incluidos: 1 2 3 4 5 6 7 8 9 10

seq(10, 1, by=-2) → Genera la secuencia de 10 hasta 1 decrementando de dos en dos, resultado: 10 8 6 4 2. En este caso el destino solo pertenecerá a la secuencia si coincide con el incremento/decremento.

seq(length=5, from=0, to=1) → Genera una secuencia desde 1 hasta 0 de 5 números distribuidos uniformemente en dicho intervalo, estos números pueden ser decimales, resultado: 0.00 0.25 0.50 0.75 1.00. Observa que el incremento es de 0,25 , pero es la propia función la que se encarga de calcularlo.

Función rep(): con esta función generamos una secuencia de números repetidos, también podemos repetir los elementos de un vector. Acepta varios parámetros que modifican su comportamiento. Por ejemplo:

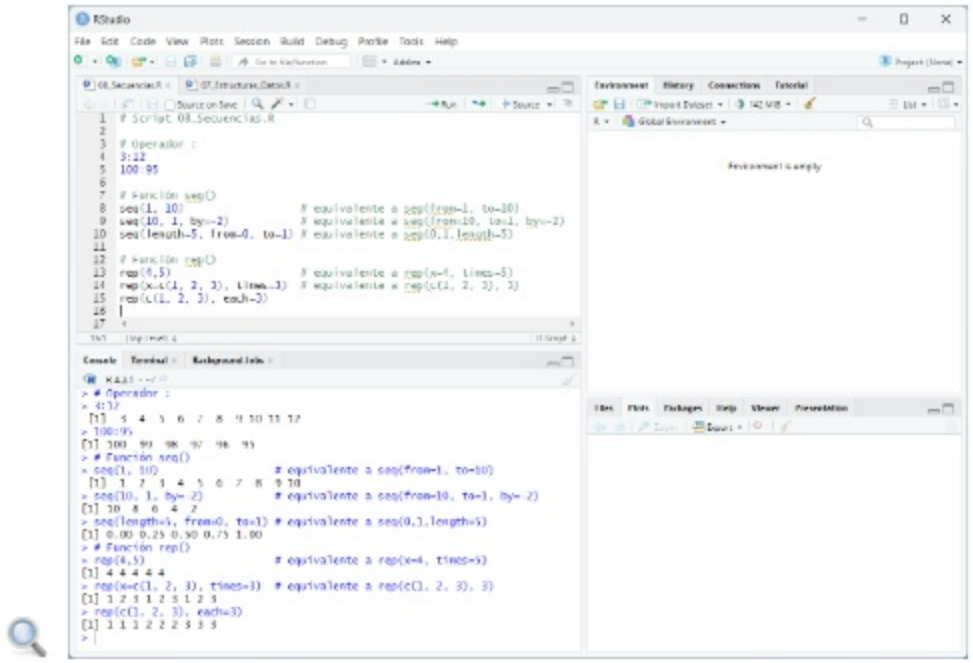
rep(4,5) → Genera una secuencia del número 4 repetido 5 veces: 4 4 4 4 4

rep(c(1, 2, 3), times=3) → Genera una secuencia repitiendo 3 veces la secuencia indicada por el vector, resultado: 1 2 3 1 2 3 1 2 3

rep(c(1, 2, 3), each=3) → Genera una secuencia repitiendo 3 veces consecutivas cada elemento del vector, resultado: 1 1 1 2 2 2 3 3 3

Como veremos mas adelante, estas secuencias nos permitirán acceder a determinados elementos de las estructuras con las que estemos trabajando, sin necesidad de recorrerlas mediante bucles, lo que lleva a una considerable mejora de la eficiencia de los algoritmos.

A continuación la captura de la ejecución del script "**08_Secuencias.R**". Abre, ejecuta y modifica este script, prueba a generar tus propias secuencias.



(Elaboración propia). *Captura de RStudio*

Concatenaciones

La función c() es una de las más utilizadas en R. Se usa para definir el contenido de otras estructuras más complejas, especificar sus dimensiones, acceder a su contenido, etc. Esta función genérica combina sus argumentos para formar un vector, convirtiendo todos los argumentos a un tipo común y secuenciándolos para formar un conjunto ordenado de objetos.



Imagen de Freepik (Licencia Gratuita Freepik)

Definición de vectores: por defecto, la salida de la función c() será un vector formado por los elementos indicados en los parámetros, el intérprete inferirá el tipo de datos (número, cadena o lógico), si los parámetros tienen distinto tipo se convertirán a un tipo común.

numeros <- c(1, 2, 3, 4, 5) → crea un vector numérico de una dimensión con los números indicados.

frutas <- c("manzana", "banana", "cereza") → crea un vector de cadenas

acierto <- c(TRUE, FALSE, TRUE, TRUE) → crea un vector de booleanos

mixto <- c(5, TRUE, "Aprobado", 6.2) → crea un vector compuesto por tipos de datos de distinta naturaleza, pero como deben ser de tipo común, el intérprete les aplica coerción al mejor tipo posible, en este caso a cadena.

mixto2 <- c(5, 7.9, 0, TRUE) → igualmente, pero esta vez el intérprete aplica coerción a numerico, transformando el valor TRUE en 1.

Concatenación de secuencias o vectores: Se pueden combinar varias concatenaciones, secuencias o vectores en uno solo, facilitando la manipulación y agregación de datos.

impar_par <- c(seq(1, 10, 2), seq(2, 10, 2)) → construye un vector por concatenación de dos secuencias, los números impares de 1 hasta 10 seguidos de los números pares de 1 hasta 10. Es decir: 1 3 5 7 9 2 4 6 8 10

cosas <- c(c("tomate", "lechuga"), c("mesa", "silla")) → construye un vector por concatenación de dos secuencias de cadenas.

Acceso a elementos: Podemos acceder o modificar determinados elementos utilizando esta función como índice para acceder a la posición que ocupan en el vector.

print(frutas[c(1, 3)]) → muestra por consola los elementos uno y tres del vector "frutas".

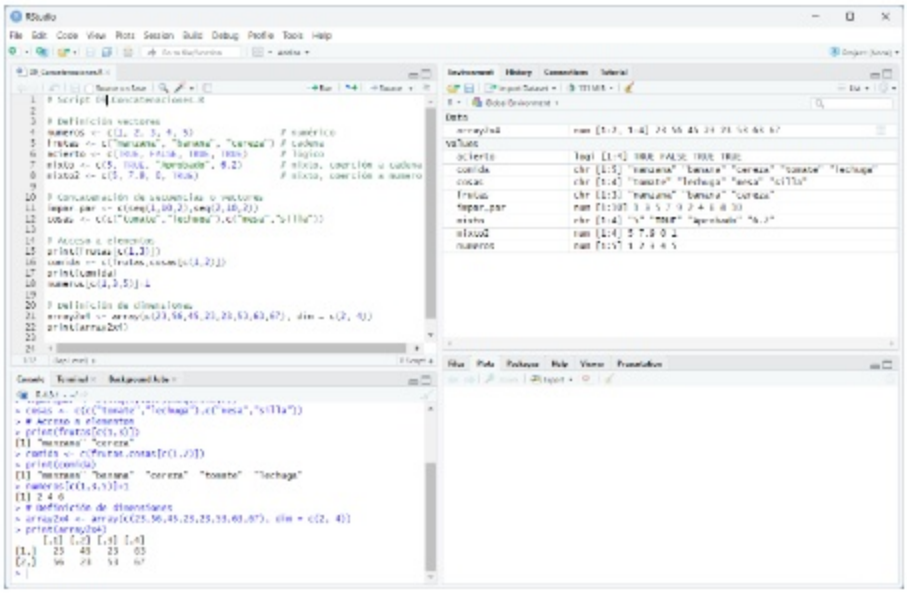
comida <- c(frutas, cosas[c(1, 2)]) → construye el vector "comida" concatenando el vector "frutas" con los elementos uno y dos del vector "cosas".

numeros[c(1, 3, 5)] + 1 → genera como salida los números que ocupan la posición 1, 3 y 5 del vector "numeros", incrementados en uno.

Definición de dimensiones: De igual forma que en los usos anteriores, podemos utilizar la salida de la función c() para dar valor a cualquier parámetro de una función, como en este caso, donde se indican las dimensiones de un vector para formar una estructura 2x4.

array2x4 <- array(c(23, 56, 45, 23, 23, 53, 63, 67), dim = c(2, 4)) → construye un vector de dos filas y cuatro columnas, distribuyendo los elementos del vector por filas.

A continuación la captura de la ejecución del script "**09_Concatenaciones.R**". La recomendación es la habitual: abre y ejecuta las sentencias del script paso a paso, observa las salidas por consola y cómo evolucionan las variables del entorno, a continuación realiza modificaciones y ejecuta tus propias variaciones.



(Elaboración propia). *Captura de RStudio*

Autoevaluación

13) ¿Qué hace la sentencia numeros[c(1,3,5)]-1?

- Construir un vector con los números 1, 3 y 5 y luego restarles 1.
- Decrementar en 1 los elementos 1, 3 y 5 del vector "numeros".
- No hace nada, la sentencia contiene un error.
- Eliminar el elemento que ocupa la última posición del vector "numeros".

14) ¿Cuál de las siguientes afirmaciones sobre el uso de la función c() es correcta?

- Solo puede combinar números.
- Puede combinar elementos de diferentes tipos, aplicando coerción.
- No permite la concatenación de secuencias.
- Solo puede combinar cadenas de texto.

1.5.1 Vectores

Los vectores son una de las estructuras de datos más importantes, se utilizan para almacenar elementos de un mismo tipo de dato, pueden ser numéricos, caracteres, lógicos, etc. Definir y manipular vectores es esencial para programar con cualquier lenguaje, también con R.

Veamos cómo definir vectores de diferentes tipos y algunas de las operaciones básicas que se pueden realizar con ellos.

Definir un vector: Para definir un vector utilizamos la función array(), especificando los objetos que lo formarán, para lo que se suele utilizar la función c(). La ejecución de la función c(), por defecto, genera como salida un array por lo que podemos omitir el uso de la función array(). Podemos definir vectores de elementos de cualquiera de los tipos básicos de datos.



Imagen de Freepik generada con IA (CC BY)

v1 <- array(c(1:5, rep(10, 2), 5:1)) → Definimos un vector unidimensional de enteros, que estará formado por tres secuencias: los enteros del uno al cinco, dos dieces y por último los enteros del cinco al uno. Es decir: 1 2 3 4 5 10 10 5 4 3 2 1.

v2 <- array(v1, dim=c(2, 6)) → Vector 2x6 formado a partir del vector anterior. Los objetos de v1 se distribuirán por columnas en el nuevo vector de dos dimensiones.

Acceso a Elementos: Podemos acceder a elementos específicos de un vector utilizando corchetes `[]`, y sabiendo la posición que ocupan, de esta forma podemos consultar o modificar su valor.

v1[c(6,7)] <- 0 → Asignamos un cero a las posiciones 6 y 7 del vector v1.

V2[2,5] <- 100 → Asignamos el valor 100 al elemento de la fila dos y columna cinco.

V2[,3:4] <- 0 → Asignamos el valor 0 a todos los elementos de las columnas tres y cuatro, observa que no ponemos nada en la posición de la primera dimensión (fila), de esa forma la operación se aplicará a todas las filas de dichas columnas.

Operaciones Aritmeticológicas: Podemos aplicar operaciones aritméticas o lógicas a todos los elementos del vector en una sola instrucción, obtendremos un vector de resultados.

v3 <- v1 * 2 → Multiplicamos por dos todos los números del vector v1 y guardamos el resultado en el vector v3.

Operaciones Aritmeticológicas Elemento a Elemento: Podemos realizar operaciones aritméticas o lógicas entre vectores de las mismas dimensiones, dichas operaciones se realizarán entre elementos que ocupan la misma posición en los distintos vectores.

v3 <- v3 – v1 → Restamos un vector a otro, elemento a elemento.

Concatenación de Vectores: Para unir o concatenar dos o más vectores utilizaremos la función c(), indicando como parámetros los vectores a unir. Obtendremos un vector con todos los elementos en el orden indicado.

v3 <- c(v1,v3) → Creamos un vector por concatenación de los vectores v1 y v3, guardamos el resultado en el vector v3.

Longitud del Vector: Para calcular la longitud de un vector utilizamos la función length().

print(paste("La longitud del vector es: ",**length(v3)**)) → Calculamos la longitud del vector v3 y la mostramos con un texto por consola.

Funciones de Agregación y Estadísticas: R proporciona funciones integradas para realizar cálculos estadísticos y agregaciones sobre vectores. Aquí algunas instrucciones donde se calcula la media, suma de todos los elementos, desviación estándar, el máximo del vector y el mínimo. Todas las funciones de agregación se muestran por consola acompañadas por un texto informativo.

Siendo v1 = 1 2 3 4 5 0 0 5 4 3 2 1

print(paste("La media de v1 es", **mean(v1)**)) → "La media de v1 es 2,5"

print(paste("La suma de v1 es", **sum(v1)**)) → "La suma de v1 es 30"

print(paste("La desviación estándar de v1 es", **sd(v1)**)) → "La desviación estándar de v1 es 1.78376517003169"

print(paste("El máximo de v1 es", **max(v1)**)) → "El máximo de v1 es 5"

print(paste("El mínimo de v1 es", **min(v1)**)) → "El mínimo de v1 es 0"

Funciones de Ordenación: Nos permiten ordenar los elementos de un vector. Disponemos de dos funciones esenciales, sort() que devuelve los elementos del vector ordenados de menor a mayor, y order() que devuelve el índice de los elementos ordenados, no los propios elementos sino sus posiciones, es decir, nos devuelve un vector de indexación de los elementos. Ambas funciones disponen de varios parámetros, entre ellos uno que nos permite ordenar descendentemente (decreasing = TRUE).

Función sort:

v1 <- c(20, 30, 10, 40)
v2 <- sort(v1) → Guarda en v2 los elementos ordenados de v1, es decir 10 20 30 40.

Función order:

v1 <- c(20, 30, 10, 40)

v3 <- order(v1) → Guarda en v3 la posición de los elementos ordenados de v1, es decir 3 1 2 4.

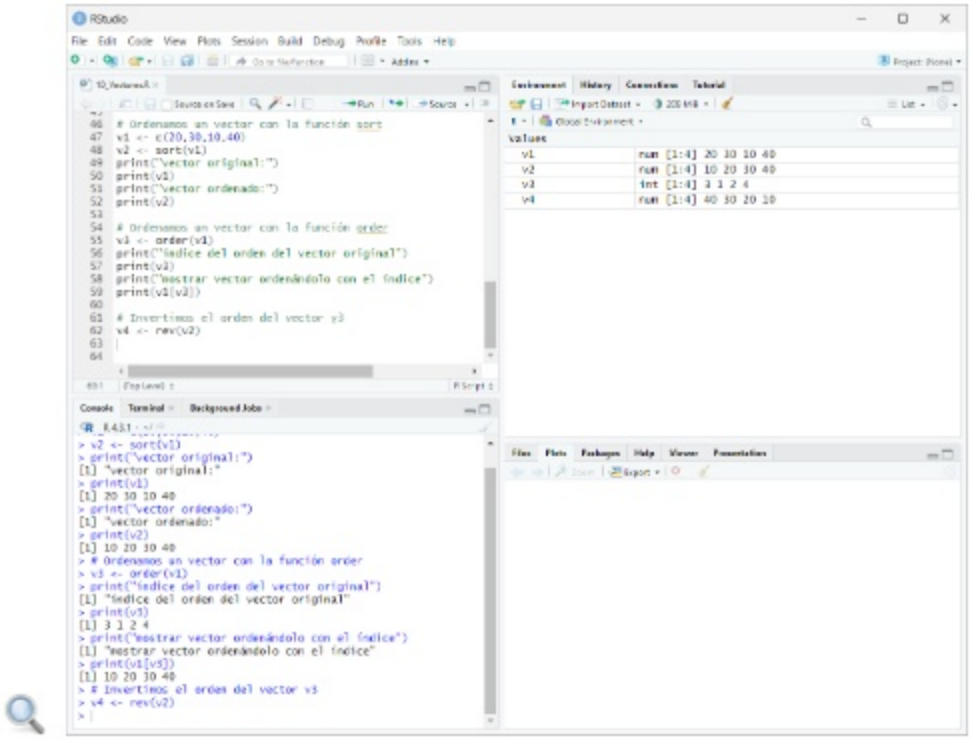
print(v1[v3]) → Mostraría el vector v1 de forma ordenada, aunque internamente los valores del vector v1 siguen en la misma posición.

Funciones de Inversión: Nos permiten invertir la posición de los elementos de un vector.

v4 <- rev(v2) → Guarda en v4 los elementos de v2 pero en el orden inverso al que ocupan en v2. Es decir 40 30 20 10

En el script "**10_Vectores.R**" puedes encontrar todas las instrucciones de ejemplo de este apartado, pruébalas y haz tus propias combinaciones. Puedes consultar los manuales y páginas recomendadas en el apartado 1 para investigar sobre distintos parámetros de estas funciones.

En la captura nos hemos centrado en los últimos ejemplos, observa como v2 contiene los elementos de v1 ordenados, en cambio v3 contiene las posiciones para que los elementos de v1 estén ordenados. Realmente es un índice que contiene el orden y podemos utilizarlo para acceder de forma ordenada a los elementos de v1.



(Elaboración propia). *Script de RStudio*

Este es un ejemplo de la potencia de R, nos va a permitir recorrer estructuras, seleccionar elementos, operar con ellos individual o conjuntamente, etc. Todo ello sin utilizar estructuras de control de flujo, que aumentan el coste de ejecución de los algoritmos.

La función which(), que veremos a continuación, va en esta línea, nos permite aplicar condiciones a los elementos de una estructura sin necesidad de utilizar estructuras de control.

Autoevaluación

15) ¿Qué función se utiliza para calcular la media de los elementos de un vector?

- sum()
- mean()
- median()
- average()

16) ¿Qué función devolvería los elementos ordenados de un vector?

- sort()
- rank()
- order()
- arrange()

Debes conocer la función which():

La función which() devuelve las posiciones de los elementos de una estructura que cumplen una determinada condición lógica. Se puede usar con vectores, matrices o dataframes y sobre cualquier tipo de dato básico (número, cadena o lógico).

Es una herramienta extremadamente relevante, se suele utilizar para el filtrado de datos o el análisis exploratorio, por ejemplo, localizar valores perdidos.

Veamos un par de ejercicios donde utilizaremos esta función para filtrar datos:

Ejercicio 1: Sustituir los valores perdidos por ceros

numeros <- c(4, 3, NA, 21, NA, 4, 3, 6, NA, 8) → Vector de números con valores perdidos.

posNA <- which(is.na(numeros)) → Usamos which() para encontrar las posiciones de los valores perdidos, ayudandonos de la función is.na() que devolverá TRUE si el valor del parámetro es NA.

print(posNA) → Mostramos las posiciones encontradas, serán tres: 3 5 9.

numeros[posNA] <- 0 → Reemplazamos los valores perdidos por ceros

numeros[posNA] <- 0 → Reemplazamos los valores perdidos por cero.

print(numeros) → Mostramos el vector sin valores perdidos.

Ejercicio 2: Sustituir los valores perdidos por la media del resto de valores del vector.

numeros <- c(4, 3, NA, 21, NA, 4, 3, 6, NA, 8) → Redefinimos el vector original.

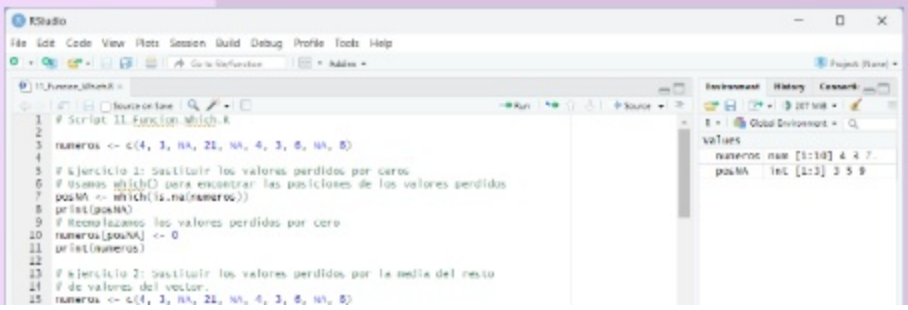
posNA <- which(is.na(numeros)) → Volvemos a extraer las posiciones de los NA.

numeros[posNA] <- mean(numeros, na.rm = TRUE) → Esta vez en lugar de reemplazar su valor por cero, lo reemplazamos por la media del resto de elementos, que no sean NA (na.rm = TRUE).

print(numeros) → Mostramos el vector sin valores perdidos.

El lenguaje R es muy rico en funciones, estas simplifican tareas y facilitan la programación, podríamos intentar realizar el ejercicio anterior sin utilizar el parámetro **na.rm** de mean (que calcula la media sin tener en cuenta los valores NA), o incluso sin utilizar la función **is.na()** pero el algoritmo sería mucho mas complejo y costoso.

A continuación puedes ver una captura con el resultado de la ejecución del script "**11_Funcion_Which.R**". En este mismo script también encontrarás la solución a los ejercicios propuestos a continuación.





(Elaboración propia). *Captura de RStudio*

Ejercicio Resuelto

Partiremos de un conjunto de datos donde, como es frecuente. tenemos datos perdidos.

Cómo debemos tratar esos datos es una cuestión que se tratará adecuadamente y en profundidad en otros módulos del Curso de Especialización.

En ejercicios anteriores hemos sustituido esos valores perdidos por cero o por la media del vector, ahora exploraremos otra alternativa. Vamos a sustituir los valores perdidos por la media de los valores adyacentes.

Sustituye cada NA del siguiente vector por la media de los valores que tiene delante y detrás en el vector. No te preocupes por los casos especiales.

numeros <- c(4, 3, NA, 21, NA, 4, 3, 6, NA, 8)

Mostrar retroalimentación

Ejercicio Resuelto

Vamos a seguir manipulando vectores, queremos añadir un elemento en una determinada posición.

Partimos de un vector con el alfabeto y deseamos añadir la letra “ñ” a continuación de la letra “n”.

```
alfabeto <- c("a", "b", "c", "d", "e", "f", "g", "h", "i", "j", "k", "l", "m", "n",
"o", "p", "q", "r", "s", "t", "u", "v", "w", "x", "y", "z")

print(alfabeto)
```

Mostrar retroalimentación

1.5.2 Matrices

Las matrices son una estructura de datos bidimensionales, que consisten en elementos dispuestos en filas y columnas. Son particularmente útiles para el análisis numérico, la manipulación de datos y las operaciones algebraicas lineales.

1) Definición y Creación de Matrices: Veremos varias formas de hacerlo.



Pasillo de Estanterías de un almacén (CC0)

En R, una matriz es una colección de elementos del mismo tipo (numérico, lógico, carácter, etc.) organizados en una cuadrícula de filas y columnas. Las matrices se crean principalmente utilizando la función **matrix()**.

matrix(data, nrow, ncol, byrow = FALSE, dimnames = NULL)

- **data**: Un vector que contiene los elementos a incluir en la matriz.
- **nrow**: El número de filas.
- **ncol**: El número de columnas.
- **byrow**: Lógico. Si es `TRUE`, los elementos se incluyen en la matriz por filas; de lo contrario, se incluirán por columnas.
- **dimnames**: Una lista de nombres de filas y columnas. Los nombres de las dimensiones de la matriz.

matriz1 <- matrix(1:8, nrow = 2, byrow = TRUE, dimnames = list(c("F1", "F2"), c("C1", "C2", "C3", "C4"))) → Crear una matriz de 2 filas y 4 columnas llenando por filas.
print(matriz1) → Muestra la matriz.

También podemos construir una matriz utilizando otro tipo de funciones, como es el caso de **cbind()** y **rbind()**, que también están disponibles para trabajar con dataframes, que veremos en el siguiente apartado.

función cbind(): combina objetos por columnas. Toma varios vectores, matrices o dataframes como argumentos y los une, columna por columna, para formar una nueva matriz o data frame.

función rbind(): combina objetos por filas. Toma varios vectores, matrices o dataframes como argumentos y los une, fila por fila, para formar una nueva matriz o data frame.

Otras funciones como **rownames()** o **colnames()** nos permiten dar nombres a las filas y columnas de una matriz ya creada.

Veamos un ejemplo, donde creamos una matriz 3x4 a partir de tres vectores con rbind(), los nombres de las filas los coge de los nombres de los vectores, los nombres de las columnas los asignamos con colnames a partir de la concatenación de la letra c y una secuencia de 1 a 4.

v1 <- c(1,2,3,4) → Creamos los tres vectores de distintas formas.

v2 <- c(seq(10,40,10))

v3 <- c(rep(0))

matriz2 <- rbind(v1,v2,v3) → Construimos la matriz definiéndola por filas.

colnames(matriz2) <- paste0("c",1:4) → Asignamos nombre a las columnas utilizando la función paste0() y una secuencia.

print(matriz2) → Mostramos la matriz.

2) Acceso a Elementos: Se realiza de igual forma que como se hacía con los vectores.

dato <- matriz1[2, 3] → Guardamos en la variable dato el elemento en la segunda fila, tercera columna.

3) Operaciones Aritmeticológicas: Igualmente que con los vectores, permite realizar operaciones aritméticas o lógicas con operadores o entre matrices.

matriz_suma <- matriz1 + 5 → Sumar 5 a todos los elementos de la matriz.

matriz_cuadrado <- matriz1 * matriz1 → Multiplicar elemento a elemento, las dos matrices deben tener las mismas dimensiones.

Operaciones propias sobre las matrices

R proporciona de forma nativa todas las operaciones necesarias para operar sobre matrices.



Imagen gratuita de Freepik (Licencia Gratuita de Freepik)

Multiplicación Matricial: Utilizamos el operador %*% para realizar la multiplicación matricial entre dos matrices compatibles. No es objeto de esta unidad explicar en que consiste esta operación, sí decir que es una operación fundamental en diversas aplicaciones, como gráficos por computadora.

En el siguiente ejemplo se definen dos matrices cuyas dimensiones son compatibles para ser multiplicadas (2x4 y 4x2)

matriz3 <- rbind(v1=2:5,v2=rep(1,4)) → Construimos una matriz 2x4.

print(matriz3) → Mostramos la matriz 2x4.

matriz4 <- rbind(a1=c(1,0),a2=c(0,1),a3=c(1,0),a4=c(0,1)) → Construimos una matriz 4x2.

print(matriz4) → Mostramos la matriz 4x2.

matriz_producto <- matriz3%*%matriz4 → Calculamos la multiplicación matricial, el resultado será una matriz 2x2.

print(matriz_producto) → Mostramos la matriz resultado.

Transposición de una Matriz: Esta operación consiste en cambiar filas por columnas.

matriz_transpuesta <- t(matriz1) → Calcula la matriz transpuesta de matriz1 y la guarda en "matriz_traspuesta"

Matriz Identidad y Diagonal de una matriz: Para generar la matriz identidad utilizamos diag(n) donde n será la dimensión de la matriz cuadrada. Esta función también podemos utilizarla para extraer la diagonal de una matriz existente.

diag(5) → Generará la matriz identidad 5x5.

diag(matri_producto) → Extraerá la diagonal de la matriz indicada, devolviéndola como un vector.

Determinante de una Matriz: Calcula el determinante, una operación tremendamente importante que R optimiza al igual que todas las descritas.

M <- rbind(c(3,-1),c(2,-2)) → Construimos la matriz.

print(M) → Mostramos la matriz.

det(M) → Calculamos y mostramos el determinante.

Varianza de una Matriz: Calcula la varianza de una matriz.

var(M) → Calculamos y mostramos la varianza.

Inversa de una matriz y resolución de ecuaciones lineales: La función solve() se utiliza para resolver sistemas de ecuaciones lineales y para calcular la inversa de una matriz.

Inversa de una Matriz: Podemos obtener la inversa a una matriz pasándo por parámetro dicha matriz a la función solve(), de la forma:

A <- matrix(c(2, 1, 3, 4), nrow = 2, byrow = TRUE) → Creamos una matriz 2x2 por filas.

print(A) → La mostramos.

A_inv <- solve(A) → Calculamos la matriz inversa y la guardamos en "A_inv".

print(A_inv) → Mostramos la matriz inversa, que también será 2x2.

Resolver Sistemas de Ecuaciones Lineales: Si tienes un sistema de ecuaciones lineales representado en la forma Ax=b, donde A es una matriz de coeficientes y b es un vector de constantes, puedes usar solve() para encontrar el vector x, de la forma:

A <- matrix(c(2, 1, 3, -1), nrow = 2, byrow = TRUE) → Matriz de coeficientes del sistema de ecuaciones.

b <- c(8, -1) → Vector de constantes, a la izquierda del igual.

x <- solve(A, b) → Calcula el valor de las variables para el sistema de ecuaciones.

print(x) → Mostramos el resultado.

En el script "**11_Matrices.R**" puedes encontrar todas las instrucciones de ejemplo de este apartado, pruébalas y haz tus propias combinaciones. Puedes consultar los manuales y páginas recomendadas en el apartado 1 para investigar sobre distintos parámetros de estas funciones y algunas otras.

Ejercicio Resuelto

Queremos utilizar R para resolver el siguiente sistema de ecuaciones:

$$\begin{cases} 2x + 3y &= -1 \\ 3x + 4y &= 0 \end{cases}$$

Imagen de producción propia (Miguel Ángel López Montero)

Implementa las sentencias necesarias para definir y resolver el sistema de ecuaciones.

Mostrar retroalimentación

Investiga sobre cómo representar un sistema de ecuaciones en R, instala y activa las librerías necesarias. Por último realiza la representación.

Mostrar retroalimentación

Autoevaluación

17) ¿Cuál de las siguientes funciones se utiliza para combinar objetos por filas en R?

- cbind()
- t()
- diag()
- rbind()

18) ¿Qué operador se utiliza en R para realizar la multiplicación matricial?

- *
- %/%
- %%
- %*%

1.5.3 Data Frames

Los dataframes son las estructuras de datos más importantes y ampliamente utilizadas en R. Un dataframe es una estructura bidimensional similar a una tabla en una base de datos o una hoja de cálculo, donde cada columna puede contener diferentes tipos de datos (números, cadenas, factores, etc.). Los dataframes son esenciales para la manipulación y análisis de datos en R.

Creación de Dataframes



Portacontenedores (CC0)

Los dataframes se pueden crear a partir de vectores, listas o directamente importando datos desde archivos como CSV, Excel, entre otros. La función principal para crear dataframes es `data.frame()`.

Creación de dataframes a partir vectores

```
# Definimos los vectores que compondrán el dataframe
nombres <- c("Ana", "Luis", "Marta", "Juan")
edades <- c(23, 34, 28, 45)
ocupaciones <- c("Ingeniera", "Doctor", "Abogada", "Profesor")

# Creamos dataframe, dándole nombre a las columnas
socios <- data.frame(Nombre = nombres, Edad = edades, Ocupación = ocupaciones)
print(socios)
```

Importar Dataframes desde Archivos: Podemos importar los datos a un dataframe desde formatos como .csv, .xls, .xlsx, .otd, .txt, ... En el caso de CSV o TXT las librerías necesarias ya se encuentran disponibles por defecto en R. Para el resto de formatos debemos instalar el paquete apropiado y utilizar las funciones de dicho paquete. Es interesante estudiar en detalle las posibilidades que aporta cada paquete, ya que nos permiten indicar las filas y columnas a cargar, realizar filtrados, etc.

En el siguiente ejemplo vamos a importar desde un archivo CSV, para ello utilizamos la función **read.csv()** que se encuentra en R por defecto, no necesitamos instalar ningún paquete.

```
df <- read.csv("ruta/clientes.csv")
print(df)
```

Acceso a Elementos del Dataframe: Podemos acceder a los elementos de un dataframe utilizando el nombre de la columna, índices o funciones específicas.

Elemento <- socios[2, 3] → Accedemos a un elemento específico, en este caso a la fila dos y columna tres.

nombres <- socios$Nombre → Accedemos a una columna específica, en este caso la columna 'Nombre'.

primera_fila <- socios[1,] → Accedemos a una fila específica, indicamos la primera fila y nada en la columna, de esta forma accedemos a todas las columnas.

Manipulación de los Datos del Dataframe

Añadir columnas: Podemos añadir columnas a dataframes ya creados, con datos extraídos de otras fuentes o calculados a partir de los propios del dataframe.

socios$Salario <- c(50000, 60000, 70000, 80000) → Añadir una nueva columna llamada 'Salario'.

socios$SalarioNeto <- socios$Salario-(0.21*socios$Salario) → Añadir una nueva columna llamada 'SalarioNeto', calculada restándole el 21% a la columna 'Salario'.

Filtrar datos: Podemos aplicarle filtros a los registros de un dataframe, obteniendo un nuevo dataframe con los registros (filas) que cumplen una determinada condición.

socios_mayores_30 <- subset(socios, Edad > 30) → Filtrar filas según una condición, mayores de 30 años.

print(socios_mayores_30) → Mostramos las filas que cumplen dicha condición.

Ordenar los datos del dataframe: Para ordenar los datos del dataframe, hacemos uso de la función order() vista cuando estudiamos los vectores. A esta función le pasamos la columna por la que queremos ordenar, nos devolverá un índice de los elementos ordenados, que utilizaremos para acceder a los datos del dataframe.

socios_ordenado <- socios[order(socios$Edad),] → Ordenamos por edad.

print(socios_ordenado) → Ordenamos por edad.

Otras funciones de interés

A continuación se muestra un listado de las funciones que nos ayudarán a trabajar con los dataframes:



Caja de Herramientas (CC0)

str(socios) → Ver la estructura del dataframe, columnas, tipos de datos, etc.

dim(socios) → Ver las dimensiones del dataframe, número de filas y columnas.

nrow(socios) → Número de filas.

ncol(socios) → Número de columnas.

summary(socios) → Ver un resumen estadístico del dataframe.

head(socios) → Ver las primeras filas.

tail(socios) → Ver las últimas filas.

names(socios) → Ver los nombres de las columnas.

Son muchas las funciones de las que disponemos para trabajar con dataframes, además podemos utilizar otros paquetes que expanden la funcionalidad básica, mejorando la presentación, añadiendo funciones que transforman los datos, permitiendo exportarlos a distintos formatos para su presentación, etc. Te invito a explorar sobre el mundo de los dataframes.

Tampoco olvides que todas las funciones vistas y ejercicios propuestos, puedes encontrarlos en los materiales proporcionados, para este apartado te interesan los scripts "**13_Dataframes.R**", y "**13b_Dataframes.R**".

Autoevaluación

19) ¿Cómo se llama la función utilizada para importar dataframes desde un archivo CSV en R?

- read_csv()
- import.csv()
- read.csv()
- load.csv()

Ejercicio Resuelto

Tenemos un conjunto de datos sobre temperatura, humedad y precipitaciones por meses, almacenados en un dataframe. De la forma:

```
temp = c(20.37, 18.56, 18.4, 21.96, 29.53, 28.16, 36.38, 36.62, 40.03, 27.59, 22.15, 19.85)

humedad = c(88, 86, 81, 79, 80, 78, 71, 69, 78, 82, 85, 83)

precipitaciones = c(72, 33.9, 37.5, 36.6, 31.0, 16.6, 1.2, 6.8, 36.8, 30.8, 38.5, 22.7)

mes = c("enero", "febrero", "marzo", "abril", "mayo", "junio", "julio", "agosto", "septiembre", "octubre", "noviembre", "diciembre")

datos = data.frame(mes = mes, temperatura = temp, humedad = humedad,precipitaciones = precipitaciones)

print(datos)
```

1) Calcula la temperatura media de los seis primeros meses del año.

Mostrar retroalimentación

2) Muestra el resumen de temperatura y humedad de los tres meses mas lluviosos.

Mostrar retroalimentación

Debes conocer

En ocasiones podemos necesitar conjuntos de datos para explorar la funcionalidad de una herramienta o incluso para desarrollar nuestro software sin trabajar directamente con datos sensibles. Aquí es donde entra Mockaroo, este sitio nos permite generar grandes datasets con una amplia gama de conjuntos de datos, indicando rangos de valores, porcentaje de valores perdidos, etc. Además, permite generar los resultados como hojas de cálculo, JSON o incluso SQL.

1.6 Estructuras de Control de Flujo

Las estructuras de control son fundamentales para dirigir el flujo de ejecución de un programa. Las principales estructuras de control son las condicionales y los bucles. Por las características de R podremos implementar nuestras aplicaciones sin necesidad de recurrir continuamente a este tipo de estructuras, pero habrá casos donde su uso sea imprescindible.



Imagen de Freepik generada por IA (Licencia Gratuita Freepik)

1. Estructuras Condicionales

Las estructuras condicionales se utilizan para ejecutar diferentes 🍋 bloques de código según se cumplan determinadas condiciones.

Estructura if-else: Se utiliza cuando se debe verificar si se ejecutará un bloque de código u otro en función de unas condiciones.

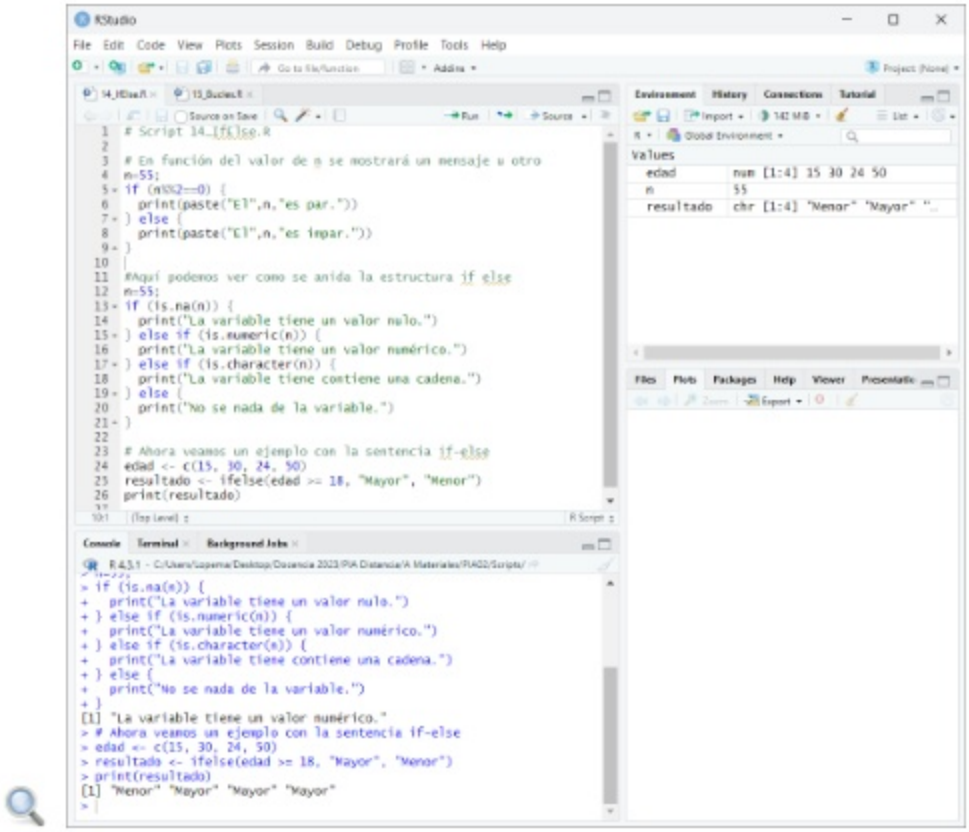
```
if (Condición Uno) {
  # Código que se ejecutará si la Condición Uno es verdadera
} else if (Condición Dos) {
  # Código que se ejecutará si la Condición Uno es falsa y la Dos verdadera
} else {
  # Código que se ejecutará si la Condición Uno y Dos son falsas
}
```

Imagen de producción propia

Sentencia ifelse: Se utiliza para aplicar condiciones de forma vectorial, especialmente útil para grandes conjuntos de datos.

```
ifelse(condición, valor_si_verdadero, valor_si_falso)
```

Veamos algunos ejemplos del uso de estas estructuras, los podrás encontrar en el Script "**14_IfElse.R**". Como siempre, te animo a modificar y probar tus propias estructuras.



(Elaboración propia). *Captura de RStudio*

El resultado de la primera sentencia, como 55%2=1, saldrá el mensaje "El 55 es impar".

En la siguiente sentencia se verifica si la variable es NA, numérica o carácter, mostrando el mensaje adecuado. En este caso, como n=55, "La variable tiene un valor numérico"

En la última sentencia partimos de un vector de enteros al que se le aplica una sentencia condicional que en función de si se verifica una condición para cada elemento del vector, generará una u otra salida. Por lo tanto el resultado será un vector con los resultados de la ejecución para cada elemento.

2. Bucles

Los bucles se utilizan para repetir la ejecución de un bloque de código.

Bucle for: Se utiliza para iterar sobre elementos de un vector, lista o cualquier otra estructura iterable. El número de ejecuciones es conocido a priori, se ejecutará el cuerpo del bucle para cada elemento de la estructura.

```
for (variable in secuencia) {
  # Código a ejecutar en cada iteración
}
```

Imagen de Producción Propia

Bucle while: Se utiliza cuando no se conoce de antemano el número de iteraciones necesarias, pero se tiene una condición que debe cumplirse.

```
while (condición) {
  # Código a ejecutar mientras la condición sea verdadera
}
```

Imagen de Producción Propia

Bucle repeat – break: Se ejecuta el cuerpo del bucle de forma infinita hasta que la salida sea provocada por la ejecución de la sentencia break.

```
repeat {
  # Código a ejecutar
  if (condición_de_salida) {
    break
  }
}
```

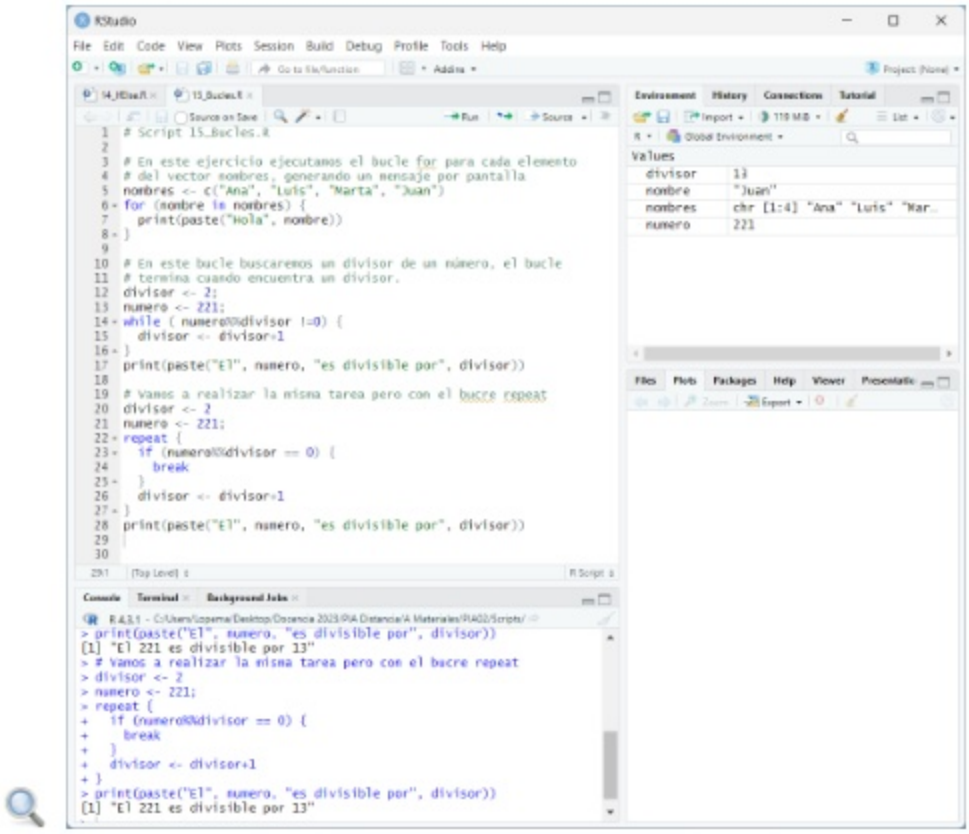
Imagen de Producción Propia

Sentencia next: Se utiliza dentro de los bucles para omitir la iteración actual y pasar a la siguiente, útil para saltar condiciones específicas.

```
for (variable in secuencia) {
  if (condición) {
    next
  }
  # Código a ejecutar en cada iteración si la condición no se cumple
}
```

Imagen de Producción Propia

En el script "**15_Bucles.R**" puedes ver algunos ejemplos de estas estructuras de control de flujo.



(Elaboración propia). *Captura de RStudio*

Para saber más

Si quieres conocer en profundidad y con ejemplos, estas y otras estructuras de control en R, te aconsejo que explores la web de R Coder.

Pregunta de Elección Múltiple

20) ¿Qué hace la sentencia next dentro de un bucle for en R?

- Termina el bucle.
- Salta a la última iteración del bucle.
- Omite la iteración actual y pasa a la siguiente.
- Reinicia el bucle.

1.7 Funciones

Las funciones son bloques de código reutilizables diseñados para realizar una tarea específica. Gracias a la implementación de funciones conseguimos un código modular, dando lugar a programas más limpios y manejables.

En R, las funciones se definen utilizando la palabra clave "function". La sintaxis básica es la siguiente:

```
nombre_funcion <- function(argumentos) {
        # Cuerpo de la función
        return(valor_de_retorno)
}
```

Imagen de Elaboración Propia

Argumentos

Las funciones pueden aceptar uno o más argumentos. Los argumentos son valores que se pasan a la función cuando se la llama.

Argumentos con Valores por Defecto: Puedes definir valores por defecto para los argumentos. Si no se proporciona un valor, se utiliza el valor por defecto:

```
saludo <- function(nombre = "Amigo") {
    mensaje <- paste("Hola,", nombre)
    return(mensaje)
}
print(saludo("Ana"))
```



Avión de Carga (CC0)

Número de Argumentos Variable: Las funciones pueden aceptar un número variable de argumentos utilizando `...`. Esto es útil cuando no sabes de antemano cuántos argumentos se van a pasar.

```
suma_varios <- function(...) {
    numeros <- c(...)
    resultado <- sum(numeros)
    return(resultado)
```

```
}
print(suma_varios(1, 2, 3, 4, 5))
```

Funciones como Argumentos: Puedes pasar nombres de funciones como argumento a otras funciones. Esto es útil para operaciones más complejas.

```
operar <- function(a, b, funcion) {
    return(funcion(a, b))
}
suma <- function(a, b) {
    return(a + b)
}
multiplicacion <- function(a, b) {
    return(a * b)
}
print(operar(4, 2, suma))
print(operar(4, 2, multiplicacion))
```

Ejercicio Resuelto

Recuerda el ejercicio resuelto del apartado 1.5.1. Donde queríamos sustituir los valores perdidos de un vector por la media de los valores adyacentes, se proponía este algoritmo:

```
numeros <- c(4, 3, NA, 21, NA, 4, 3, 6, NA, 8)

posNA <- which(is.na(numeros))

numeros[posNA] <- (numeros[posNA-1]+numeros[posNA+1])/2

print(numeros)
```

Como se advertía, este algoritmo no contemplaba casos especiales, es decir, no funciona correctamente cuando:

- los valores NA se encuentren en los extremos del vector, ya que cuando accedamos a sus vecinos, no existirán.
- existan valores NA seguidos, un NA tendrá como vecino otro NA y el cálculo de la media no será posible.
- también hay que tener en cuenta que las dos situaciones anteriores concurran al mismo tiempo.

En aquel momento nos faltaban herramientas para resolver este problema, ahora tenemos todo lo que necesitamos. Diseña un algoritmo que resuelva el problema.

Solo ten en cuenta una consideración, cuando un NA tenga un solo vecino, se igualará al vecino.

Mostrar retroalimentación

Condiciones y términos de uso de los materiales

Materiales desarrollados inicialmente por el Ministerio de Educación, Cultura y Deporte y actualizados por el profesorado de la Junta de Andalucía bajo licencia Creative Commons BY-NC-SA.

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL
Junta de Andalucía

Antes de cualquier uso leer detenidamente el siguente Aviso legal

Historial de actualizaciones

Versión: 02.00.01	Fecha de actualización: 03/09/24
Actualización de materiales y correcciones menores.	

Versión: 02.00.00	Fecha de actualización: 19/06/24	Autoría: Miguel Ángel López Montero
Ubicación: Completa **Mejora (tipo 3):** Esta unidad, tal como está planteada, trata de explorar las distintas plataformas de Inteligencia Artificial pero descuida la parte más importante sobre como se explotan estos servicios mediante el acceso a las APIs, que apenas se trata. Se propone dividir los contenidos de esta unidad en dos partes, la primera sobre Plataformas de IA, se rehace completamente y se trataría en la Unidad 1, dándole un enfoque más práctico a la hora de realizar una comparativa entre plataformas. Una segunda parte, sobre el uso de APIs, se trataría en la Unidad 3, se abordaría su uso real una vez trabajados los lenguajes y librerías de IA. Cuestión que actualmente no es tenida en cuenta. Se propone rehacer completamente la unidad, para abordar en detalle los lenguajes de programación de IA, haciendo especial hincapié en las características y posibilidades de los lenguajes de desarrollo de IA, así como la elección del lenguaje para cada tarea específica, sin olvidar la integración de los distintos lenguajes. En esta unidad se tratarían los entornos de desarrollo e IDEs que actualmente se tratan en la Unidad 3, pero se le daría un enfoque más racional acorde a su peso en los resultados de aprendizaje, en lugar de dedicarle una unidad completa en detrimento de otros contenidos mas relevantes. Además, se propone abordar la introducción al lenguaje de programación R y los lenguajes de marcado, contenidos con un fuerte peso en los criterios de evaluación de este módulo y que no se contemplan en el temario. Contenidos Propuestos: 1.- Lenguajes de programación de IA. 2.- Entornos de Desarrollo. 3.- Lenguaje de Programación R: importación, estudio y preparación de datos. 4.- Lenguajes de marcado: HTML, XML y JSON y Markdown **Ubicación:** Completa **Mejora (Examen online):** Ajustar a contenidos propuestos. **Ubicación:** Completa **Mejora (Mapa conceptual):** Ajustar a contenidos propuestos. **Ubicación:** Completa **Mejora (Orientaciones del alumnado):** Ajustar a contenidos propuestos.		

Versión: 01.00.00	Fecha de actualización: 05/07/23
Versión inicial de los materiales.	